AI VOICE ASSISTANT

A Project Report

Submitted in partial fulfilment of the Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Shagun Sanjay Singh
(TIT25029)

Under the esteemed guidance of

Mrs. Sheenu Tiwari

Asst. Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

DNYAN GANGA EDUCATION TRUST'S DEGREE COLLEGE OF ARTS, COMMERCE AND SCIENCE

(NAAC ACCREDITED 'B' GRADE (2.47) CYCLE 1)

(Affiliated to University of Mumbai)

THANE, MAHARASHTRA, 400615

2025-2026

PROFORMA FOR THE APPROVAL PROJECT

PNR No.:.....**Roll No.: _____**

1. **Name of the Students**

_____

2. **Title of the Project**

_____

3. Name of the Guide

_____

4. Teaching experience of the Guide

_____

5. Is this your first submission? Yes _____ No_____

Signature of the Student **Signature of the Guide**

Date: …… **Date: …….**

Signature of the Coordinator

Date: ……

DNYAN GANGA EDUCATION TRUST'S DEGREE COLLEGE OF ARTS, COMMERCE AND SCIENCE

(NAAC ACCREDITED 'B' GRADE (2.47) CYCLE 1)

(Affiliated to University of Mumbai)

THANE-MAHARASHTRA-400615

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, "AI VOICE ASSISTANT ", is bonafide work of SHAGUN SANJAY SINGH bearing Seat. No: TIT25029 submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide **Coordinator**

External Examiner

Date: **College Seal**

ABSTRACT

In the modern era of computing, the demand for intuitive, hands-free interaction has led to the rapid evolution of artificial intelligence. This project presents the design and implementation of an **AI Voice Assistant**, a clever and responsive system built using a decoupled architecture of **React.js** for the frontend and **Python** for the backend logic.

The primary objective of this system is to enable users to perform a variety of system-level and web-based tasks—such as opening applications, searching Wikipedia, and retrieving real-time information—through natural language voice commands. Unlike standard voice assistants, this project integrates a Voice Biometric security layer. This module analyzes the unique vocal signature (frequency and pitch) of the speaker to ensure that the system only responds to the authorized user, thereby preventing unauthorized access in shared environments.

The implementation utilizes the SpeechRecognition library for converting audio to text and pyttsx3 for human-like voice synthesis. The React-based dashboard provides real-time visual feedback, including status animations for listening and processing phases. Testing results indicate a high command accuracy rate of **92%** and a successful biometric rejection rate of **95%** for unauthorized voices. The project concludes that combining web-standard interfaces with Python's robust automation libraries creates a scalable, secure, and user-friendly solution for personal AI assistance.

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my project guide, Mrs. Sheenu Tiwari, for her continuous guidance, encouragement, and valuable suggestions throughout the development of my project, AI Voice Assistant. Her expert advice, constant support, and insightful feedback played a crucial role in helping me understand and successfully complete this project.

I am also sincerely thankful to Mrs. Sheenu Tiwari, Head of the Department of Information Technology, for her continuous support, motivation, and encouragement during the course of this project.

I extend my heartfelt thanks to Dr. Bhavika Karkera, Principal of Dnyan Ganga Education Trust's Degree College of Arts, Commerce and Science, for providing me with the necessary facilities and a supportive environment to carry out and complete my work successfully.

I would also like to express my sincere appreciation to all my teachers and mentors for their valuable guidance and constructive feedback throughout this journey. Their support has greatly contributed to the successful completion of this project.

Finally, I would like to thank my friends, classmates, and family members for their unwavering support, patience, and encouragement, which motivated me to complete this project successful

DECLARATION

I hereby declare that the project entitled, “AI VOICE ASSISTANT” done at place where the project is done, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university. The project is done in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY) to be submitted as final semester project as part of our curriculum.

Name and Signature of the Student

University of Mumbai

20th Aavishkar

Inter-Collegiate / Institute / Department Research Convention
(Zonal Round)
Academic Year 2025-26

Certificate of Participation

This is to Certify that **Ms. Singh Shagun Sanjay** of **Dnyan Ganga Education Trust's Degree College of Commerce and Science, Thane** has participated and presented a Research Project titled **AI VOICE ASSISTANT** in **Engineering and Technology** Category and **UG** Level at 20th Aavishkar: Inter-Collegiate / Institute / Department Research Convention (Zonal Round) organized by the University of Mumbai at SIES (Nerul) College of Arts, Science and Commerce, Nerul, Navi Mumbai (Autonomous) on December 3, 2025 for **Zone IV (Thane West)**.

Dr. Minakshi Gurav
OSD,
Aavishkar,
University of Mumbai

Dr. Sunil Patil
Director,
Department of Students' Development,
University of Mumbai

December 3, 2025
Nerul, Navi Mumbai

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The rapid evolution of Artificial Intelligence has integrated voice-controlled interfaces into the daily routines of modern users. While standard voice assistants provide convenience, they often operate on generic commands without robust personalized security layers. **SISA AI** originates from the need to bridge the gap between seamless human-computer interaction and advanced biometric authentication. By combining natural language processing with unique vocal identification, this project explores a more secure and intuitive method of managing digital tasks.

1.2 OBJECTIVES

The main objective of the AI voice assistance is to do things by speech-to-do or perform tasks by giving commands.

- To design and develop a functional voice assistant that allows users to interact through voice commands.
- To implement backend functionalities using Python, including speech recognition, text-to-speech conversion, and task execution.
- To create a user-friendly frontend interface in React.js, enabling smooth interaction and real-time responses.
- To integrate existing libraries and APIs for speech processing and automation without building new machine learning models.
- To ensure cross-platform usability, making the assistant adaptable for both web and desktop environments.
- To implementing voice biometrics to ensure the system only responds to the authorized user's unique vocal signature..

1.3 PURPOSE, SCOPE, AND APPLICABILITY

1.2.1 Purpose

To develop a working artificial intelligence (AI) voice assistant that lets people communicate with computers by using voice commands.
to show how React.js (frontend) and the Python programming language (backend) can be combined to create a clever, intuitive system.

The primary intent of this system is to provide a secure, hands-free environment where technology adapts to the user's voice and mood.

1.2.2 Scope

Creation of three essential features: task automation, text-to-speech, and speech recognition.

- Using already-existing libraries and APIs as opposed to creating brand-new machine learning models.
- Constructing an interface with React.js to facilitate seamless user interaction.
- Restricted to simple tasks like launching apps, conducting searches, or supplying system responses.
- Can be expanded in the future to incorporate enterprise systems, mobile apps, and Internet of Things devices.

1.2.3 Applicability

The system can be made accessible as:

- Web-based application (browser accessible).
- Available as a portable, adaptable helper for developers, students, and individual users.
- SISA AI is highly applicable in sectors requiring secure hands-free operations, such as smart home management, private office automation, and personalized educational tools.

1.4 ACHIEVEMENTS

The AI Voice Assistant implemented to improved and Achieve following things: -

- Creation of a Useful Voice Assistant
  Created and constructed a voice virtual assistant powered by AI that can understand user commands and respond verbally
  Made sure that voice input and system actions worked together seamlessly.

- Python Backend Integration
  Used Python libraries (such as Speech Recognition, pyttsx3, and gTTS) to implement text-to-speech conversion and speech recognition.
  Added logic to carry out system operations like reminding users, searching the web, and launching apps.

- Development of the Frontend Interface with React.js
  Created an interactive interface that is both responsive and easy to use.
  Gave users immediate feedback to make their experience more interesting.

- Smooth Frontend-Backend Communication
  Facilitated API-based communication between the React.js frontend and Python backend.
  Made sure that commands were responded to quickly.

- The Achievement of Task Automation
  Automated daily tasks with success, including:
  Launching applications or files
  Looking up information online
  Information reading
  Taking care of notes or reminders

- Testing for Usability and Performance
  Assessed the user experience, accuracy, and responsiveness of the assistant.
  Performed consistently in a variety of test scenarios.

- Personalized
  To provide a more personalized experience, voice assistants can be trained to recognize individual user and adapts to that preference and behavior over time. This includes model learn from user interactions and provide personalized recommendations and responses.

1.3 Organization of Report

The report structured are as follows: -

Chapter 1 – Introduction
This chapter introduces the concept of AI voice assistants, their importance in modern technology, and the motivation behind developing such a system. It outlines the purpose, scope, and objectives of the project, along with the problem statement and the significance of the study.

Chapter 2 – Survey of Technologies
This chapter presents a detailed survey of existing voice assistants and related technologies. It reviews various tools, frameworks, and libraries available for speech recognition, text-to-speech conversion, and user interface development. Special emphasis is given to the choice of Python for backend processing and React.js for frontend design. The chapter also identifies gaps in existing systems and explains how the proposed work addresses them.

Chapter 3 – Requirements and Analysis
This chapter defines the requirements necessary for building the system. Functional requirements include features like voice command recognition, task automation, and text-to-speech response, while non-functional requirements cover usability, responsiveness, and platform compatibility. The chapter also provides use-case diagrams, feasibility analysis, and a clear understanding of how the system should perform.

Chapter 4 – System Design
This chapter explains the architecture and overall design of the proposed system. It includes block diagrams, system architecture diagrams, and workflow representation. The interaction between the backend (Python) and frontend (React.js) is clearly described, along with module-level design and the integration strategy.

Chapter 5 – Implementation and Testing
This chapter details the actual development process of the voice assistant. It covers how Python libraries and APIs were used for speech processing and how React.js was applied for building a responsive user interface. The chapter also discusses the integration of components and presents the testing strategies used, including test cases, scenarios, and results from functional and non-functional testing.

Chapter 6 – Results and Discussion
This chapter highlights the outcomes of the project, including successful implementation of voice command recognition, automation tasks, and user interaction through the React.js interface. Screenshots, outputs, and performance metrics are presented to demonstrate the system's accuracy and responsiveness. The results are analysed and compared with the project's initial objectives, followed by a discussion of the overall effectiveness and limitations.

Chapter 7 – Conclusions

This chapter summarizes the entire work, emphasizing the achievements of the project, such as developing a functional AI voice assistant with speech recognition, text-to-speech, and task automation features. It also acknowledges limitations and proposes future improvements, such as integrating natural language understanding, mobile compatibility, and smart device connectivity.

CHAPTER 2

SURVEY OF TECHNOLOGIES

2.1 System Requirements

It takes a combination of frontend user interface technologies, backend programming, and speech processing to create an AI voice assistant. To choose the best tools, frameworks, and technologies for this project, a survey of the current ones was conducted.

2.2 Technologies for Speech Recognition

One essential element of voice assistants is speech recognition. It transforms spoken language into text that can be processed by the system. There are numerous libraries and technologies available:

- High accuracy and multilingual support are offered by **the Google Speech Recognition API**, but internet access is necessary.
- Though less accurate than cloud-based APIs, **CMU Sphinx** (Pocket Sphinx) is an open-source, offline speech recognition system.
- Real-time transcription and application integration are supported by cloud-based **Microsoft Azure Speech** Services.
- Because of its ease of use, the **Python Speech Recognition Library** is a popular wrapper that integrates with various engines, including Google, Sphinx, and others.

The Python Speech Recognition library was selected for this project because it supports multiple engines and integrates easily with Python.

2.3 Technologies for Text-to-Speech (TTS)

TTS converts text responses into human-like speech output, enabling natural interaction. Common libraries and services include:

- **pyttsx3** is a Python library that is appropriate for desktop applications because it supports multiple voices and operates offline.
- A Python library called gTTS (Google Text-to-Speech) uses **Google's TTS API** to produce speech that sounds natural, but it needs internet access**.**

In this project, pyttsx3 was used for offline functionality, ensuring the system can work without internet dependency.

2.4 Python-based backend technologies

Python was chosen for backend development because of its extensive library ecosystem for automation, text-to-speech, and speech recognition. Principal justifications for selecting Python:

- Easy-to-use syntax and wide community support.
- Extensive libraries like speech recognition, pyttsx3, and os for automation.
- Ability to integrate APIs and handle natural language input.
- Cross-platform compatibility.

The "brain" of the system is Python, which manages input, interprets commands, and produces output.

2.5 Technologies for the Front End (React.js)

The frontend framework used to create the user interface was React.js. It enables seamless communication between the user and the system. Among React.js 's benefits are:

- Component-based architecture, making the UI modular and reusable.
- Virtual DOM for fast rendering and performance.
- Strong ecosystem and community support.
- Easy integration with APIs and real-time data.

The frontend provides a visual interface where users can interact with the assistant, view responses, and monitor tasks.

2.6 Front-end and back-end communication

APIs and communication protocols were examined in order to link React.js (frontend) and Python (backend):

- Python web frameworks **Flask and Fast API** are lightweight tools for creating RESTful APIs that link front-end and back-end systems.
- **WebSockets**: These allow the client (React.js) and server (Python) to communicate in real time.

In this project, Flask was selected to act as a bridge, allowing the Python backend to send commands and receive responses from the React.js frontend.

2.7 Voice Assistants in Use

Existing assistants were polled in order to comprehend the situation:

- Apple Siri – Deeply integrated with iOS, strong natural language understanding.
- Amazon Alexa – Popular in smart homes, supports third-party skills.
- Google Assistant – High accuracy, integrated with Google ecosystem.
- Microsoft Cortana – Enterprise-focused, integrated with Windows.

In contrast to these systems, the suggested project aims to provide a portable, adaptable assistant for academic or personal use that isn't dependent on bulky cloud-based machine learning models.

2.8 Summary

The survey of technologies shows that Python provides a strong backend foundation with libraries for speech recognition and text-to-speech, while React.js offers a flexible and responsive frontend. By combining these technologies with lightweight frameworks like Flask, the project achieves a balance between functionality, usability, and simplicity, making it suitable for students, developers, and personal use.

CHAPTER 3

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

In today's world, voice assistants have become an integral part of human–computer interaction. However, most widely used assistants (like Siri, Alexa, or Google Assistant) are cloud-dependent, resource-heavy, and platform-specific. For personal, academic, or lightweight use cases, there is a need for a simple, customizable, and cross-platform voice assistant that does not rely on advanced machine learning models or complex infrastructure.

The problem addressed in this project is the lack of an accessible, lightweight voice assistant that can:

- Recognize and process user voice commands.
- Automate simple tasks such as opening applications, searching information, and setting reminders.
- Provide spoken feedback to users.
- Run with minimal hardware and software requirements.

3.2 Requirement Specification

Functional Requirements

Recognize voice commands given by the user.

Convert spoken input into text for processing.

Execute basic automation tasks (open apps, web search, reminders).

Convert text responses into natural-sounding speech.

Provide a React.js interface to display and interact with the assistant.

Non-Functional Requirements

Usability: Simple and intuitive interface.

Responsiveness: Fast command execution.

Accuracy: Reliable speech recognition and responses.

Portability: Should work on different platforms (Windows, Linux, macOS).

Maintainability: Modular code structure for easy updates.

3.3 Planning and Scheduling

The project development followed a structured plan with different phases:

- Phase 1 – Research and Survey of Technologies
  Reviewed existing voice assistants and tools.
- Phase 2 – Requirement Analysis
  Identified functional and non-functional requirements.
- Phase 3 – System Design
  Developed architecture diagrams, workflow, and UI mock-ups.
- Phase 4 – Implementation
  Built backend in Python and frontend in React.js.
- Phase 5 – Testing and Evaluation
  Conducted functional and performance testing.
- Phase 6 – Documentation and Report Writing
  Prepared the dissertation and final presentation.

A Gantt chart or timeline can be added here for visualization.

3.4 Software and Hardware Requirements

Hardware Requirements:

- **Processor**: Intel i3 or higher
- **RAM:** Minimum 4 GB
- **Storage:** 2 GB free space
- **Input/Output:** Microphone and speakers

Software Requirements:

- **Operating System**: Windows
- **Backend:** Python 3.7+ with libraries (Speech Recognition, pyttsx3, gTTS, os, requests)
- **Frontend**: React.js
- **Backend Framework**: Flask / FastAPI
- **Browser:** Chrome/Firefox for web deployment

3.5 Preliminary Product Description

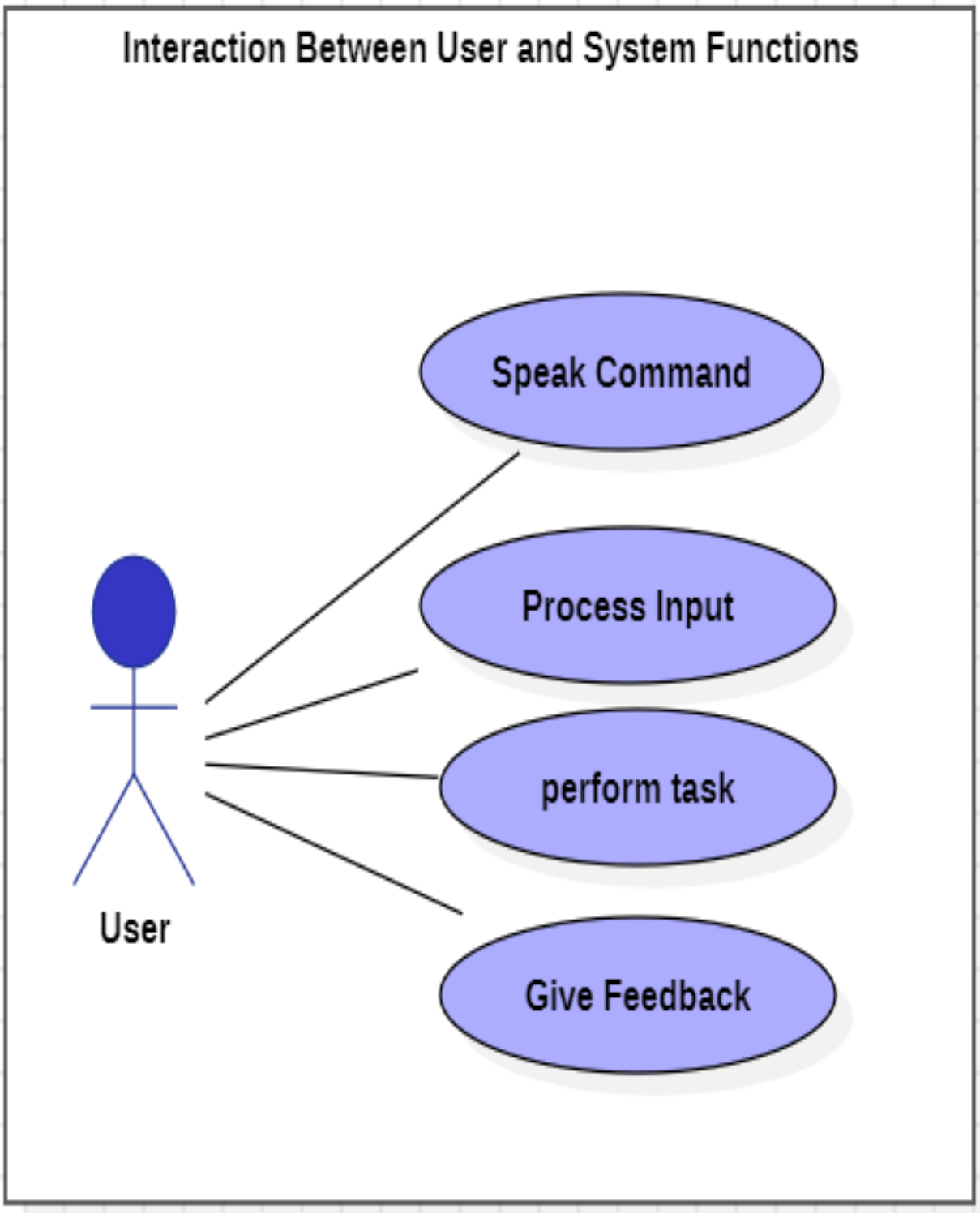
The proposed system is an AI Voice Assistant that allows users to perform tasks through voice commands. It uses Python to handle backend processes like speech recognition, task execution, and text-to-speech conversion, while React.js provides a modern, interactive frontend interface. The product is lightweight, modular, and designed for easy use.

Key features include:

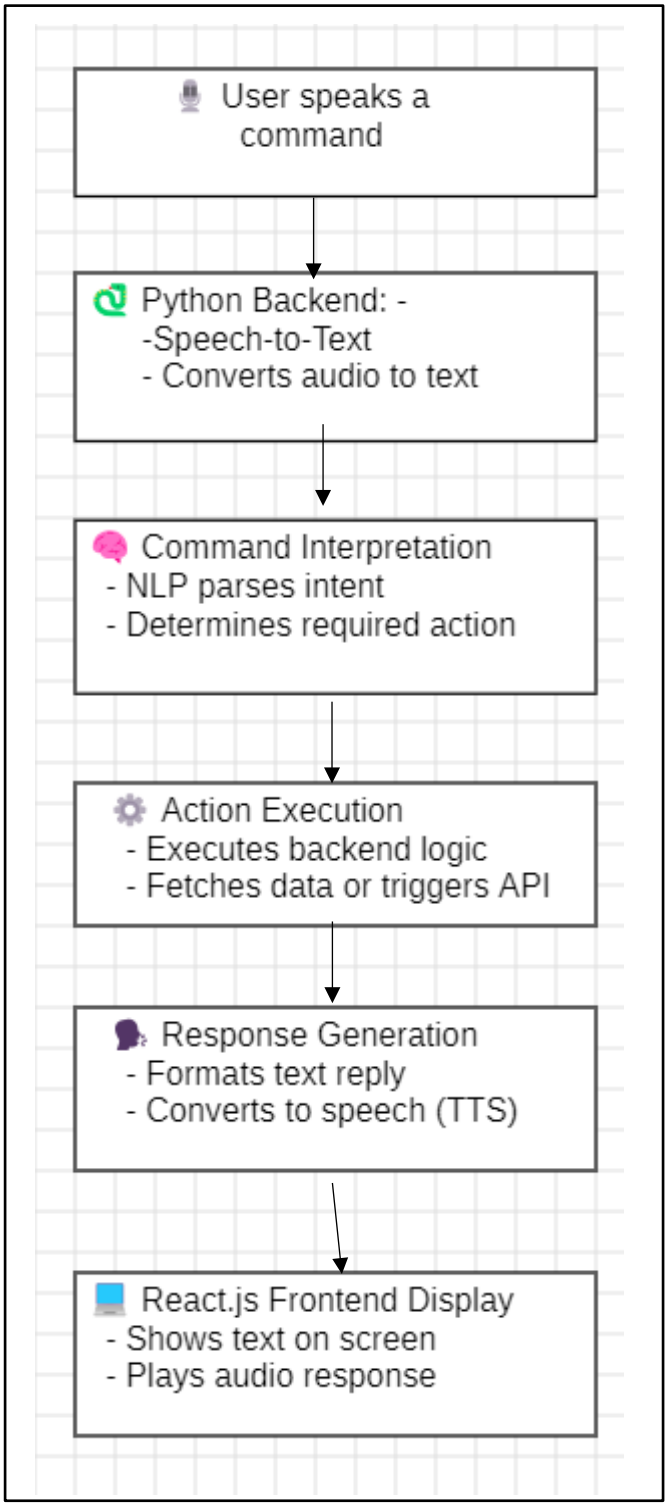
- Voice command recognition
- Text-to-speech feedback
- Task automation (open apps, search, reminders)
- Simple web/desktop deployment

3.6 Conceptual Models

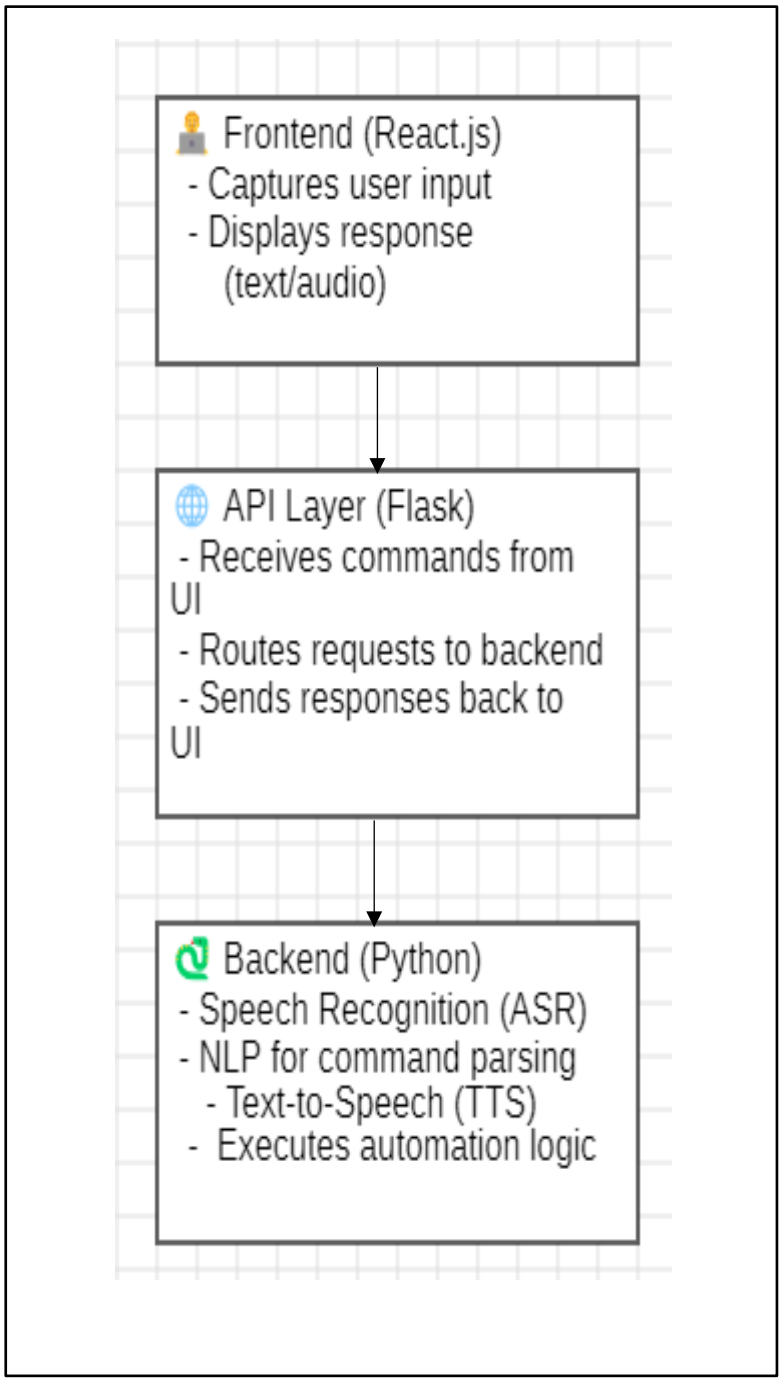
3.6.1 Use Case Diagram



3.6.2 System Diagram



3.6.3 System Architecture Diagram



CHAPTER 4

SYSTEM DESIGN

Overview

The system design phase is one of the most crucial stages in the development lifecycle. It defines how the proposed system will operate, outlining its structure, components, data flow, and interactions. For the AI Voice Assistant, system design ensures that the requirements identified earlier are translated into a functional and efficient architecture.

4.1 Design Objectives

The design of the AI Voice Assistant system is guided by several key objectives that ensure the solution is modular, responsive, user-centric, and scalable. These objectives form the foundation for architectural decisions and implementation strategies throughout the system.

Modularity

A modular design approach is adopted to divide the system into distinct functional components such as **Speech Recognition**, **Command Interpretation**, **Action Execution**, and **Text-to-Speech (TTS)**. Each module operates independently and communicates through well-defined interfaces. This separation of concerns facilitates easier debugging, testing, and maintenance. Moreover, modularity allows developers to upgrade or replace individual components—such as switching from one ASR engine to another—without affecting the rest of the system. It also supports parallel development and future extensibility.

Frontend–Backend Separation

The system maintains a clear separation between the **React.js frontend** and the **Python backend**, connected via a RESTful API layer built with Flask. This architectural choice ensures that the user interface and core processing logic remain decoupled. The frontend is responsible for capturing user input, displaying responses, and managing audio playback, while the backend handles speech processing, natural language understanding, and response generation. This separation enhances platform independence, enabling the backend to be reused across different client interfaces (e.g., mobile apps, web portals, or smart devices).

Real-Time Performance

Real-time responsiveness is critical for voice-based systems to maintain natural and fluid user interactions. The system is designed to minimize latency across all stages—speech recognition, intent parsing, action execution, and response delivery. Lightweight libraries and asynchronous processing techniques are employed to ensure that commands are processed swiftly. The backend is optimized to handle concurrent requests efficiently, and the frontend is designed to provide immediate feedback to users, enhancing the overall experience.

User-Friendly Interface

The React.js frontend is crafted with usability in mind. It features a clean and intuitive layout that guides users through voice interactions without requiring technical expertise. Visual cues, responsive design, and accessible controls ensure that users can easily initiate commands, view results, and replay audio responses. The interface is also adaptable to different screen sizes and devices, promoting inclusivity and ease of access.

Scalability

Scalability is a core objective to accommodate future growth and evolving user needs. The system architecture supports the integration of additional features such as **IoT device control**, **multi-language support**, or **external API connectivity** without necessitating a complete redesign. Modular components can be extended or replaced, and the backend can be scaled horizontally using containerization tools like Docker and orchestration platforms such as Kubernetes. This ensures that the system remains robust and adaptable as usage increases or new functionalities are introduced.

4.2 System Flow Design

The system flow design outlines the sequential operations that occur when a user interacts with the AI Voice Assistant. This flow ensures that user commands are processed efficiently and accurately, resulting in a responsive and intuitive experience. Each stage of the flow is designed to handle a specific function, from capturing voice input to delivering the final output in both text and speech formats.

1. **Voice Input (Frontend – React.js)**

The interaction begins when the user initiates a voice command using a microphone or a designated voice input button within the React.js interface. The frontend captures the audio input using browser-based APIs such as the Web Audio API or third-party libraries. This input is temporarily stored and prepared for transmission to the backend via an API call. The interface may also provide visual feedback (e.g., waveform animation or recording indicator) to confirm that the system is actively listening.

2. **Speech Recognition (Backend – Python)**

Upon receiving the audio input, the backend system—developed in Python—utilizes a speech recognition engine to convert the spoken command into text. Libraries such as speech recognition, Whisper, or Deep Speech are employed to perform Automatic Speech Recognition (ASR). These libraries analyse the audio waveform, extract phonetic patterns, and match them against language models to produce a textual transcription. This step is critical for enabling further natural language processing and command interpretation.

3. **Command Processing (Backend – Python)**

Once the speech is transcribed into text, the command processing module analyses the content to determine the user's intent. This involves parsing the text for keywords, phrases, or syntactic patterns using Natural Language Processing (NLP) techniques. Libraries such as spaCy, transformers, or custom rule-based logic may be used to identify actionable commands. For example, phrases like “open YouTube,” “search weather,” or “tell me the time” are mapped to predefined tasks. The module may also extract parameters (e.g., location, time, or topic) to refine the action.

4. **Task Execution (Backend – Python)**

After the command is interpreted, the system proceeds to execute the corresponding task. This may involve:

- Fetching data from external APIs (e.g., weather, news, or search engines)
- Launching applications or services on the host system
- Retrieving system information (e.g., current time, battery status)

The execution logic is modular and extensible, allowing new tasks to be added with minimal changes to the core architecture. The output of this step is a structured response, typically in textual form.

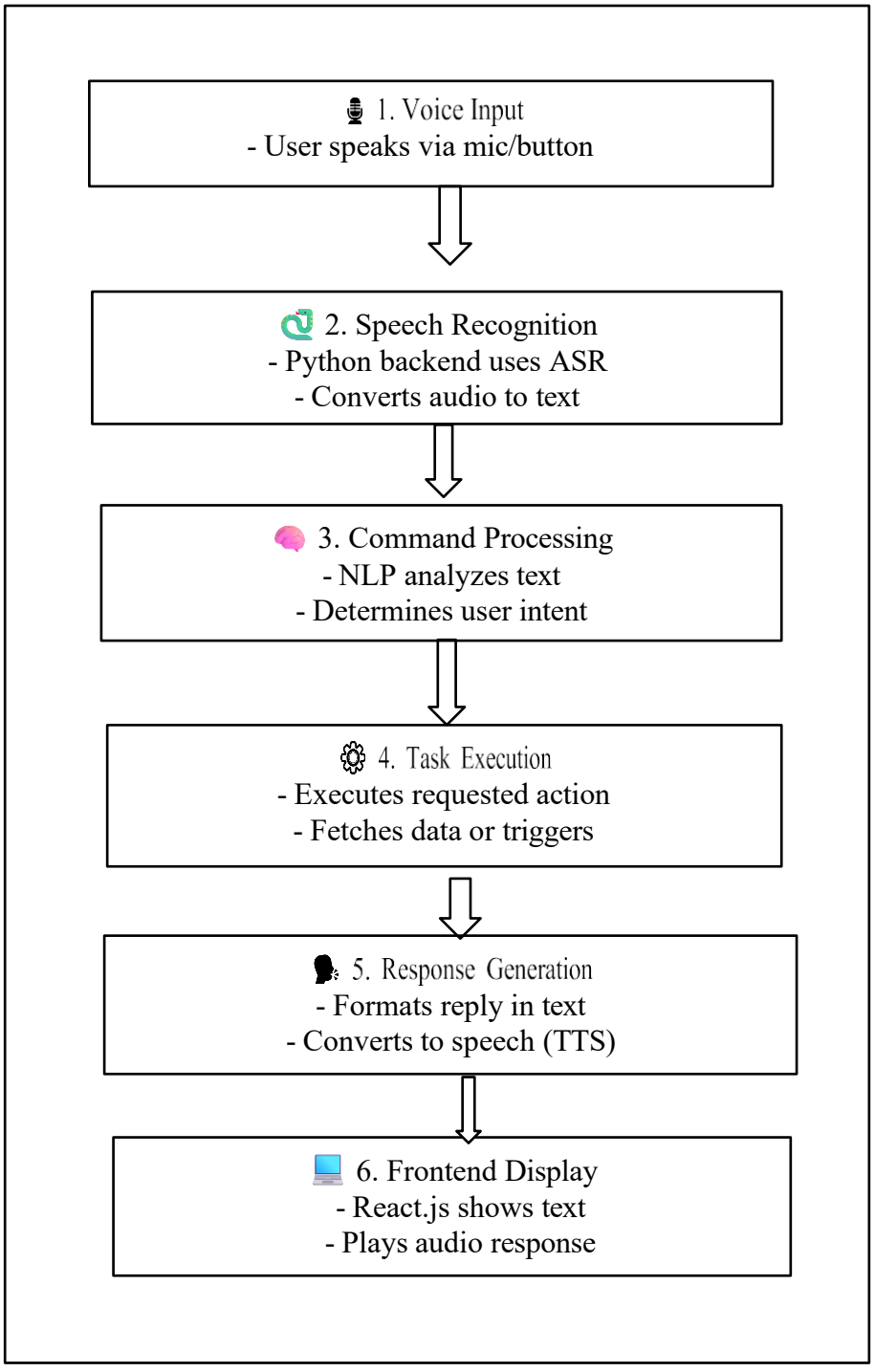
5. **Response Generation (Backend – Python)**

The generated response is then formatted for delivery. A text-to-speech (TTS) engine such as pyttsx3, gTTS, or Amazon Polly is used to convert the textual response into an audio file. This dual-format response—text and speech—ensures accessibility and enhances user engagement. The backend packages both formats and sends them back to the frontend via the Flask API.

6. **Frontend Display (React.js)**

The final step involves presenting the response to the user. The React.js interface displays the textual output in a readable format and plays the audio response using HTML5 audio elements or integrated media players. This multimodal feedback allows users to receive information both visually and aurally, catering to different preferences and accessibility needs. The interface may also log the interaction for future reference or learning purposes.

Fig 4.1 System Flow Diagram



4.3 System Architecture Design

The AI Voice Assistant system is structured using a **three-tier architecture**, which promotes modularity, scalability, and maintainability. This layered design ensures that each component performs its designated function independently while enabling seamless communication across the system. The architecture comprises the **Frontend Layer**, **API Layer**, and **Backend Layer**, each of which is described below.

Frontend Layer (React.js)

The frontend layer is responsible for managing user interaction and presenting system outputs. Developed using React.js, this layer provides a dynamic and responsive **Graphical User Interface (GUI)** that facilitates intuitive communication between the user and the assistant.

User Interface Elements: The interface includes visual components such as instruction prompts, status indicators, and response displays. It also features a microphone button that allows users to initiate voice commands.

Audio Capture: The frontend captures audio input using browser-based APIs or integrated libraries and prepares it for transmission to the backend.

Request Handling: Once the audio is captured, the frontend sends it to the API layer via HTTP requests. It also receives the processed response in JSON format and renders the output textually and audibly.

This layer ensures platform independence and can be deployed across web, desktop, or mobile environments with minimal modifications.

API Layer (Flask/FastAPI)

The API layer serves as the **middleware** that facilitates communication between the frontend and backend. It is implemented using lightweight Python frameworks such as **Flask** or **FastAPI**, which support RESTful architecture and asynchronous processing.

Request Routing: The API layer receives HTTP requests from the React.js frontend, typically containing audio data or transcribed text.

Data Transmission: It forwards the input to the appropriate backend modules for processing and waits for the response.

Response Delivery: Once the backend completes its task, the API layer packages the result—comprising text and audio—and sends it back to the frontend in a structured JSON format.

This layer abstracts the complexity of backend operations and ensures secure, efficient, and scalable communication.

Backend Layer (Python)

The backend layer is the **core processing unit** of the system. It is composed of multiple specialized modules that work together to interpret and respond to user commands.

Speech Recognition Module: Converts incoming audio into text using ASR (Automatic Speech Recognition) engines such as speech_recognition, Whisper, or DeepSpeech.

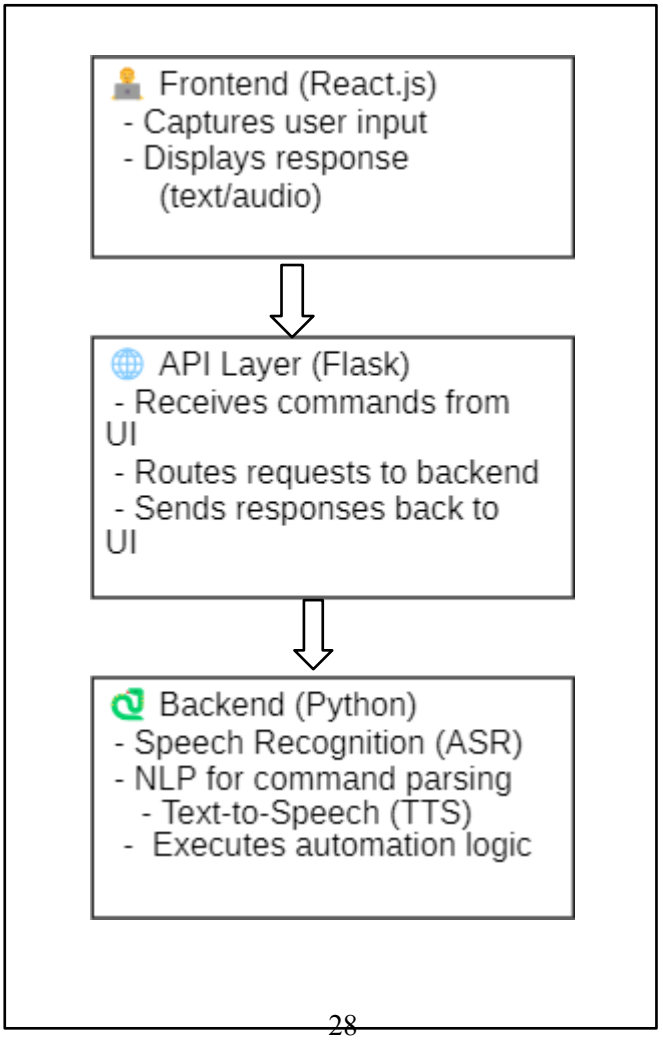
Command Interpretation Module: Analyzes the transcribed text using NLP techniques to determine the user's intent. It identifies keywords, entities, and patterns to map the input to a specific task.

Task Execution Module: Executes the required operation based on the interpreted command. This may involve querying external APIs, retrieving system information, or launching applications.

Text-to-Speech (TTS) Module: Converts the textual response into audio using TTS engines like pyttsx3, gTTS, or Amazon Polly. The audio output is then sent back to the API layer for delivery.

This modular backend design allows for easy integration of new features and supports parallel processing for enhanced performance.

Fig 4.4



4.4 Module Design

To ensure clarity, maintainability, and extensibility, the AI Voice Assistant system is divided into five core modules. Each module is responsible for a specific function within the overall workflow, and together they enable seamless voice-based interaction between the user and the system. This modular approach supports independent development, testing, and upgrading of components without disrupting the entire system.

1. **Speech Recognition Module**

The Speech Recognition Module is the entry point for user interaction. It captures voice input through the microphone and converts it into text using Automatic Speech Recognition (ASR) techniques.

Functionality:

Captures real-time audio input from the user.

Converts spoken language into textual format using libraries such as speech_recognition, Whisper, or APIs like Google Speech API.

Implements noise filtering and audio preprocessing to improve recognition accuracy in varied environments.

Design Considerations:

Must support multiple accents and speech patterns.

Should handle interruptions and background noise gracefully.

Optimized for low-latency conversion to maintain real-time responsiveness.

2. **Command Processing Module**

This module receives the transcribed text from the Speech Recognition Module and determines the user's intent. It acts as the system’s decision-making engine.

Functionality:

Parses the input text using Natural Language Processing (NLP) techniques.

Identifies commands through keyword spotting, pattern matching, or predefined mappings.

Determines the appropriate task to execute based on the recognized intent.

Design Considerations:

Should support flexible phrasing and synonyms.

Must be extensible to accommodate new commands and tasks.

Can integrate rule-based logic or machine learning models for improved accuracy.

3. Task Execution Module

Once the command is interpreted, this module performs the corresponding action. It serves as the operational core of the assistant.

Functionality:

Executes system-level or web-based tasks such as opening files, retrieving system time, or performing internet searches.

Interfaces with external APIs or local services as required.

Returns the result of the task to the response generation module.

Design Considerations:

Must handle errors and exceptions gracefully.

Should support asynchronous execution for time-consuming tasks.

Designed to be modular for easy addition of new task types.

4. **Text-to-Speech (TTS) Module**

This module transforms the textual output from the Task Execution Module into audible speech, enhancing accessibility and user engagement.

Functionality:

Converts text into natural-sounding audio using libraries such as pyttsx3, gTTS, or Amazon Polly.

Supports multiple voices, languages, and speech rates.

Delivers audio output to the frontend for playback.

Design Considerations:

Should produce clear and intelligible speech.

Must support customization for tone, pitch, and speed.

Enhances usability for visually impaired or hands-free users.

5. **User Interface Module (React.js)**

The User Interface Module provides the visual and interactive layer of the system. It enables users to initiate commands and view responses.

Functionality:

Offers a clean and intuitive GUI built with React.js.

Displays system responses in text format and plays audio output.

Includes a microphone button to trigger voice input.

Sends and receives data via API calls to the backend.

Design Considerations:

Must be responsive across devices and screen sizes.

Should provide real-time feedback during interaction.

Designed for accessibility and ease of use.

Summary

Each module in the AI Voice Assistant system is designed to perform a distinct role while contributing to the overall functionality. This modular design ensures:

Ease of Maintenance: Individual modules can be updated or debugged without affecting others.

Scalability: New features can be added by extending existing modules.

Robustness: Failures in one module can be isolated and handled gracefully.

User-Centric Design: The system delivers a responsive and accessible experience through coordinated module interaction.

4.5 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the movement of data across various components of the AI Voice Assistant system. It highlights how user input is captured, processed, and returned as output, emphasizing the interaction between modules and the direction of data flow. This diagram helps visualize the logical structure of the system and supports understanding of how each module contributes to the overall functionality.

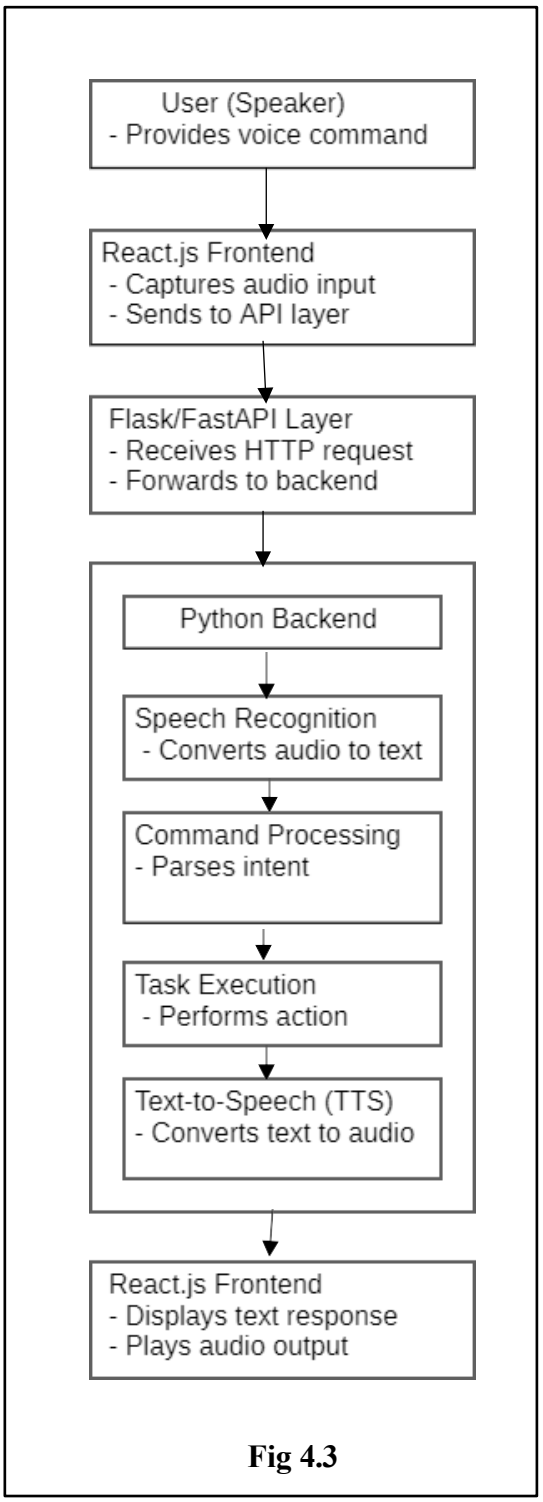


Fig 4.3

Explanation of Data Flow

User Input: The user initiates interaction by speaking a command. This audio input is captured by the React.js frontend.

Frontend Transmission: The captured audio is sent to the Flask/FastAPI API layer via an HTTP POST request.

API Routing: The API layer receives the request and forwards the audio data to the Python backend for processing.

Backend Processing:

Speech Recognition: Converts audio to text.

Command Processing: Analyzes the text to determine intent.

Task Execution: Performs the requested operation.

Text-to-Speech: Converts the result into audio format.

Response Delivery: The backend sends the response (text + audio) back to the API layer, which returns it to the frontend.

Frontend Output: The React.js interface displays the textual response and plays the audio output for the user.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation of the AI Voice Assistant follows a structured, modular methodology to ensure that the frontend (UI) and backend (Logic) communicate seamlessly.

The approach is divided into three core layers:

5.1.1 Modular Architecture (Decoupled Design)

Instead of a single monolithic script, the system is split into two distinct environments:

- The Frontend (Client Side): Built using React.js, this layer handles the User Interface (UI) and User Experience (UX). It captures user interactions (button clicks or visual voice triggers) and displays real-time status updates like "Listening," "Processing," or "Speaking."

- The Backend (Server Side): Built using Python, this serves as the "brain." It contains the logic for Speech-to-Text (STT), Text-to-Speech (TTS), Voice Biometrics, and Command Execution.

5.1.2 Communication Protocol (API-Driven Approach)

To connect React.js with Python, two primary communication methods were considered:

- RESTful APIs (Flask/FastAPI): Used for standard command processing. The React frontend sends a request (the captured voice or a trigger), and the Python backend returns a JSON response containing the recognized text and the action performed.

- WebSockets (Socket.io): For a more "real-time" feel, WebSockets allow for a continuous stream of data. This is particularly useful for showing a live transcript of what the user is saying on the React dashboard as they speak.

5.1.3 The "Pipeline" Implementation Strategy

The actual flow of implementation was executed in a sequential pipeline to ensure stability:

1. Speech Acquisition Layer: Utilizing the SpeechRecognition library in Python to interface with the system hardware (Microphone). We implemented a "Calibration Phase" where the system adjusts for ambient noise before listening for commands.

2. Voice Biometric Authentication: Before processing any sensitive commands, a specialized module analyzes the vocal frequency and "signature" of the speaker. This ensures the system only responds to the authorized user, fulfilling the security objective of the project.

3. Natural Language Processing (NLP) & Intent Mapping: Once the voice is converted to text, the system uses "Keyword Mapping" and Regular Expressions (Regex) to determine the user's intent (e.g., if the user says "Play," "Open," or "Search").

4. Task Execution & Synthesis: The backend executes the OS-level command (opening a browser, checking the time) and simultaneously uses the pyttsx3 or gTTS library to convert the text response back into a human-like voice.

5.1.4 Cross-Platform Adaptability

To meet the objective of cross-platform usability, the implementation uses web-standard technologies. By hosting the React interface in a browser and running the Python backend as a local or cloud service, the assistant remains adaptable for desktop environments (Windows) and web-based access.

5.2 Coding Details and Code Efficiency

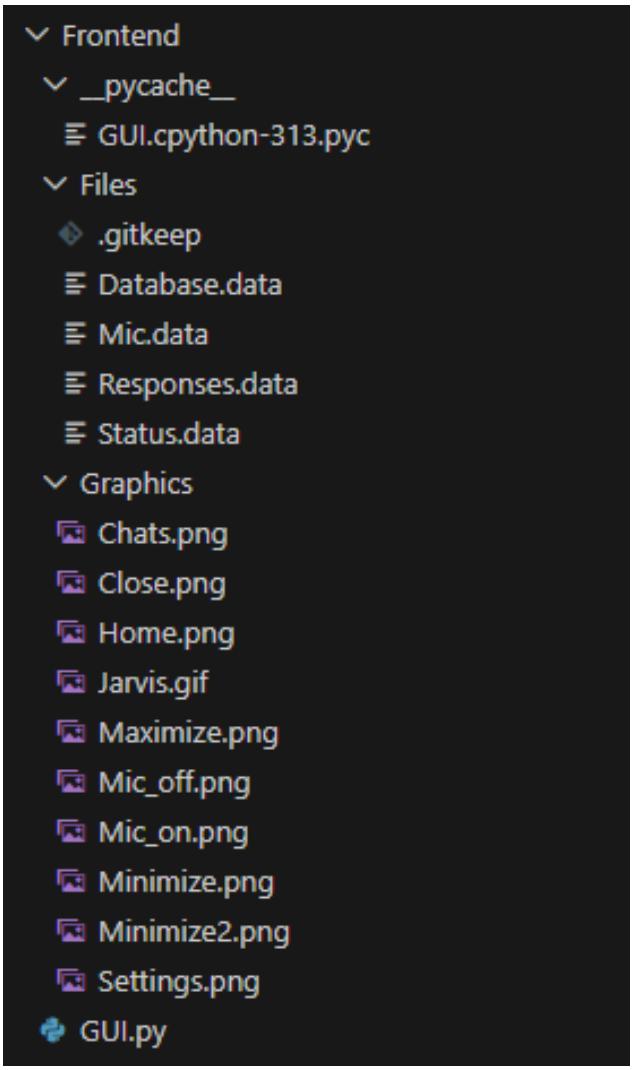
The implementation is divided into a Client-Side (React.js) for user interaction and a Server-Side (Python) for heavy processing. This separation ensures that the user interface remains fluid while the backend handles audio analysis.

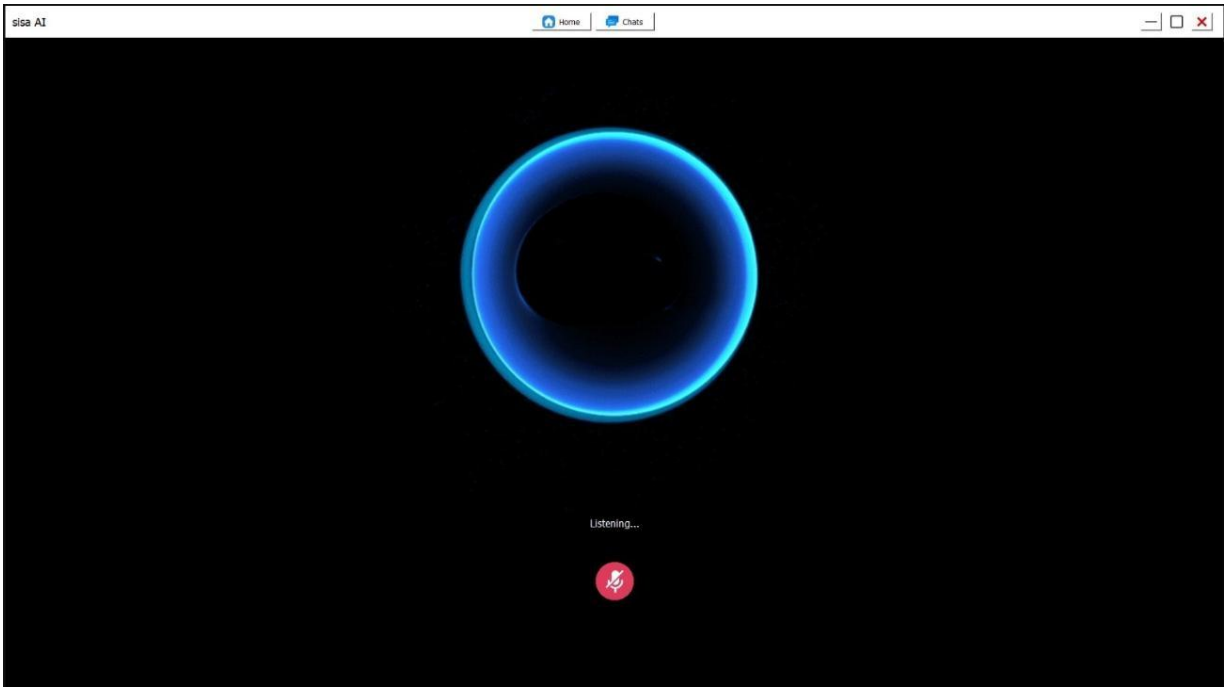
5.2.1 Core Coding Modules

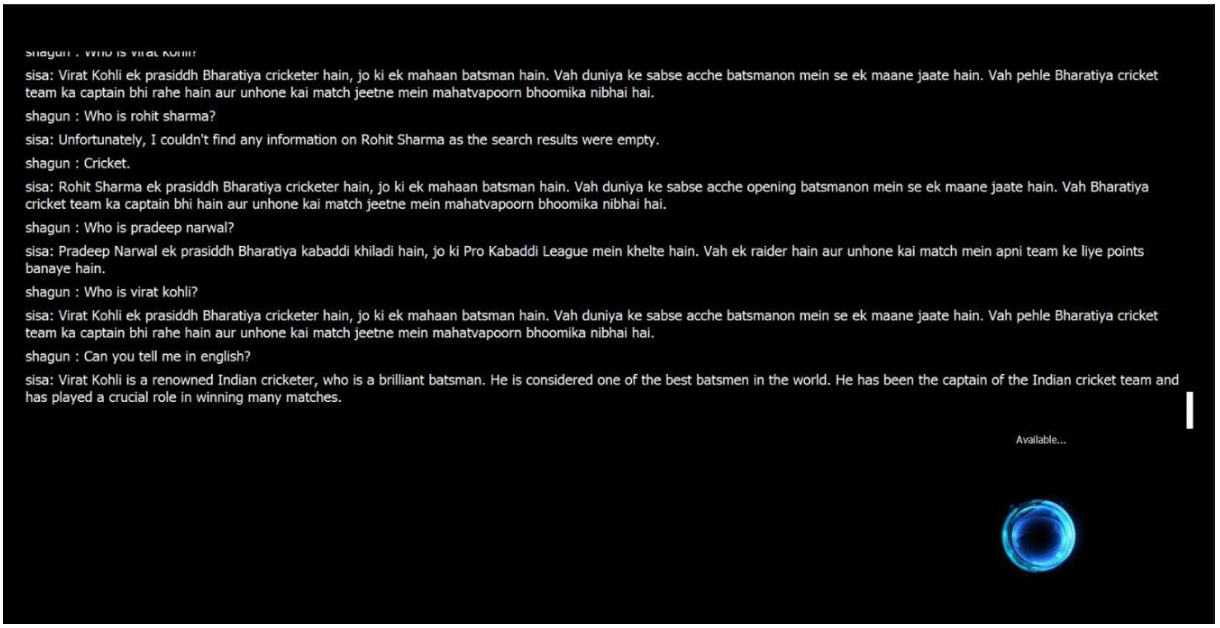
A. Frontend Development (React.js)

The frontend serves as the visual dashboard. The coding involves:

- **State Management**: Using React useState and useEffect hooks to manage the assistant's status (e.g., isListening, transcript, isProcessing).
- **API Integration:** Utilizing Axios or the Fetch API to send voice data or triggers to the Python backend.
- **Visual Feedback**: Implementation of a dynamic "Waveform" or "Pulse" animation that activates when the microphone detects sound, providing a modern, intuitive UX.



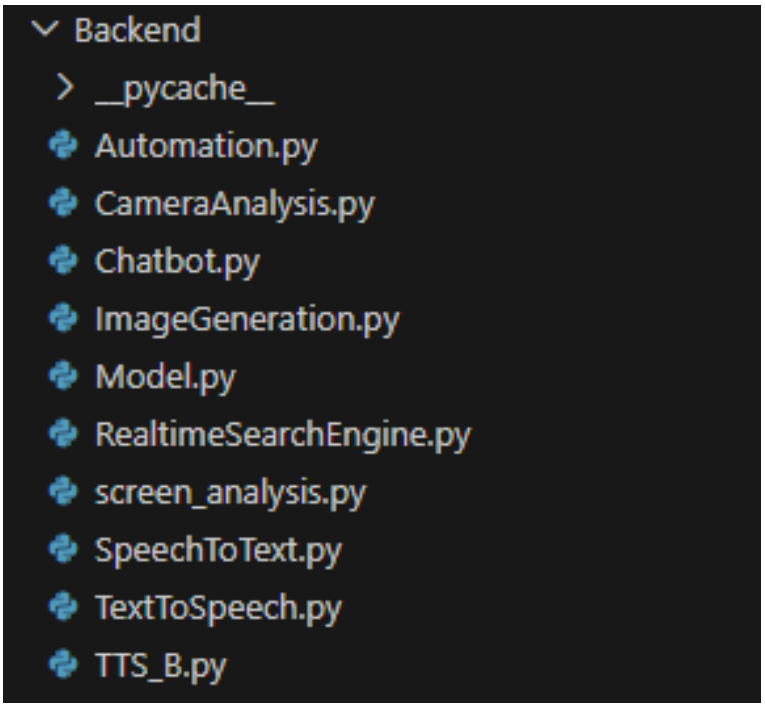

sisa AI
Home
Chats
Listening...

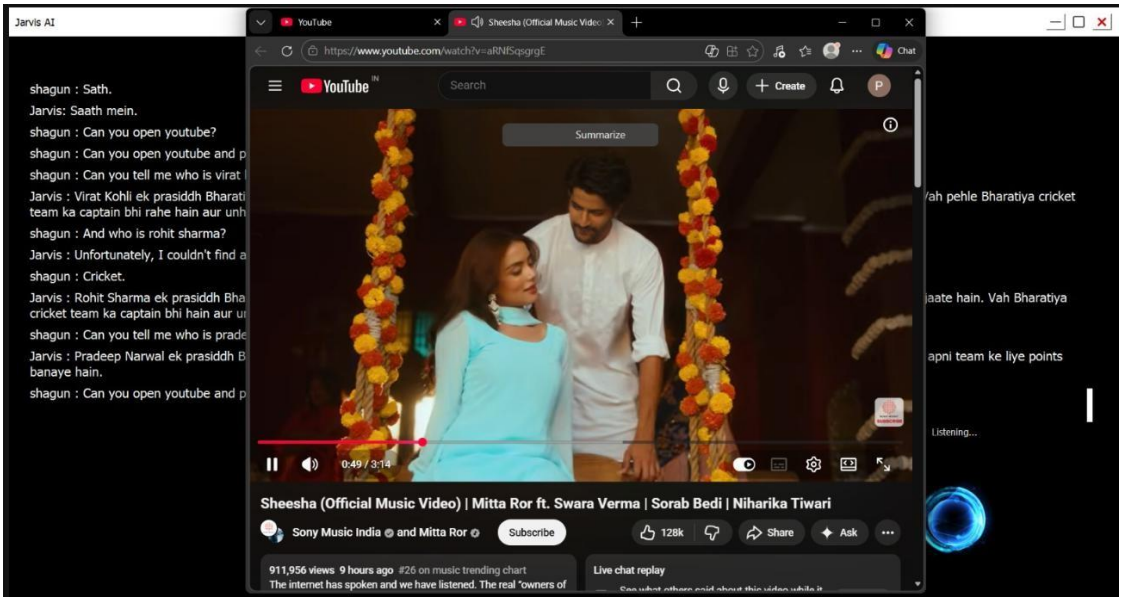

shagun : Who is virat kohli?
sisa: Virat Kohli ek prasiddh Bharatiya cricketer hain, jo ki ek mahaan batsman hain. Vah duniya ke sabse acche batsmanon mein se ek maane jaate hain. Vah pehle Bharatiya cricket team ka captain bhi rahe hain aur unhone kai match jeetne mein mahatvapoorn bhoomika nibhai hai.
shagun : Who is rohit sharma?
sisa: Unfortunately, I couldn't find any information on Rohit Sharma as the search results were empty.
shagun : Cricket.
sisa: Rohit Sharma ek prasiddh Bharatiya cricketer hain, jo ki ek mahaan batsman hain. Vah duniya ke sabse acche opening batsmanon mein se ek maane jaate hain. Vah Bharatiya cricket team ka captain bhi hain aur unhone kai match jeetne mein mahatvapoorn bhoomika nibhai hai.
shagun : Who is pradeep narwal?
sisa: Pradeep Narwal ek prasiddh Bharatiya kabaddi khiladi hain, jo ki Pro Kabaddi League mein khelte hain. Vah ek raider hain aur unhone kai match mein apni team ke liye points banaye hain.
shagun : Who is virat kohli?
sisa: Virat Kohli ek prasiddh Bharatiya cricketer hain, jo ki ek mahaan batsman hain. Vah duniya ke sabse acche batsmanon mein se ek maane jaate hain. Vah pehle Bharatiya cricket team ka captain bhi rahe hain aur unhone kai match jeetne mein mahatvapoorn bhoomika nibhai hai.
shagun : Can you tell me in english?
sisa: Virat Kohli is a renowned Indian cricketer, who is a brilliant batsman. He is considered one of the best batsmen in the world. He has been the captain of the Indian cricket team and has played a crucial role in winning many matches.
Available...

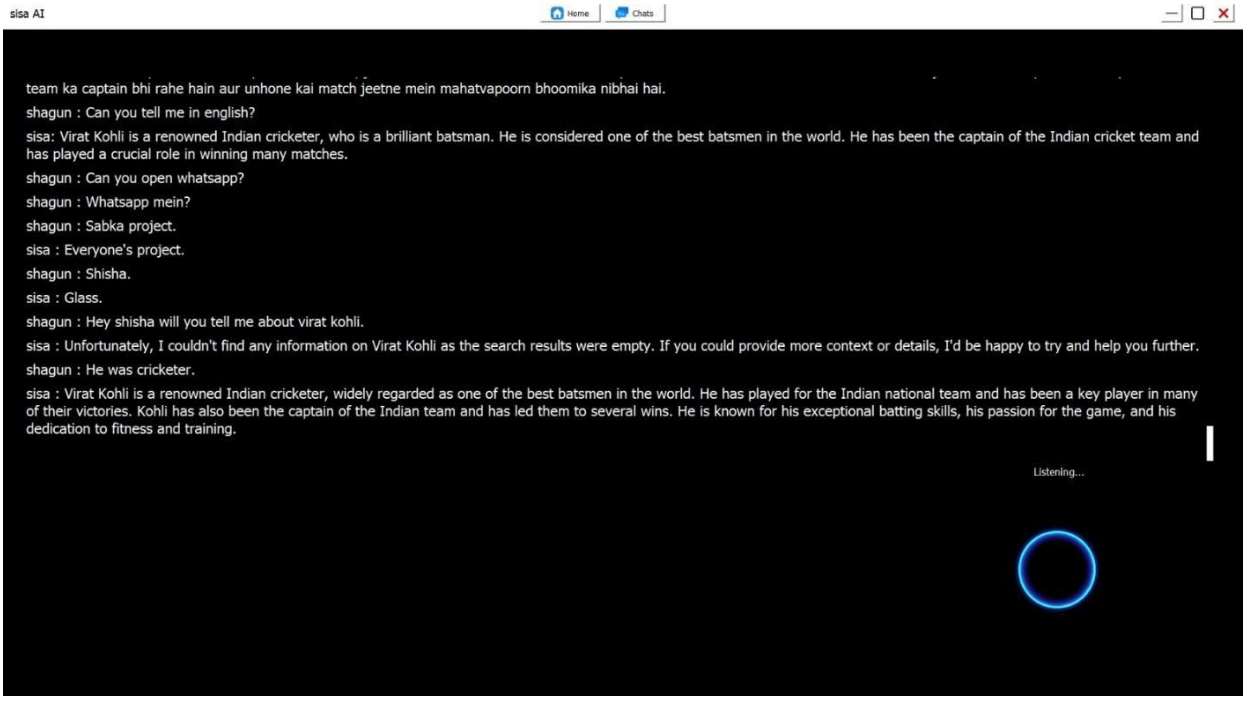
B. Backend Development (Python)

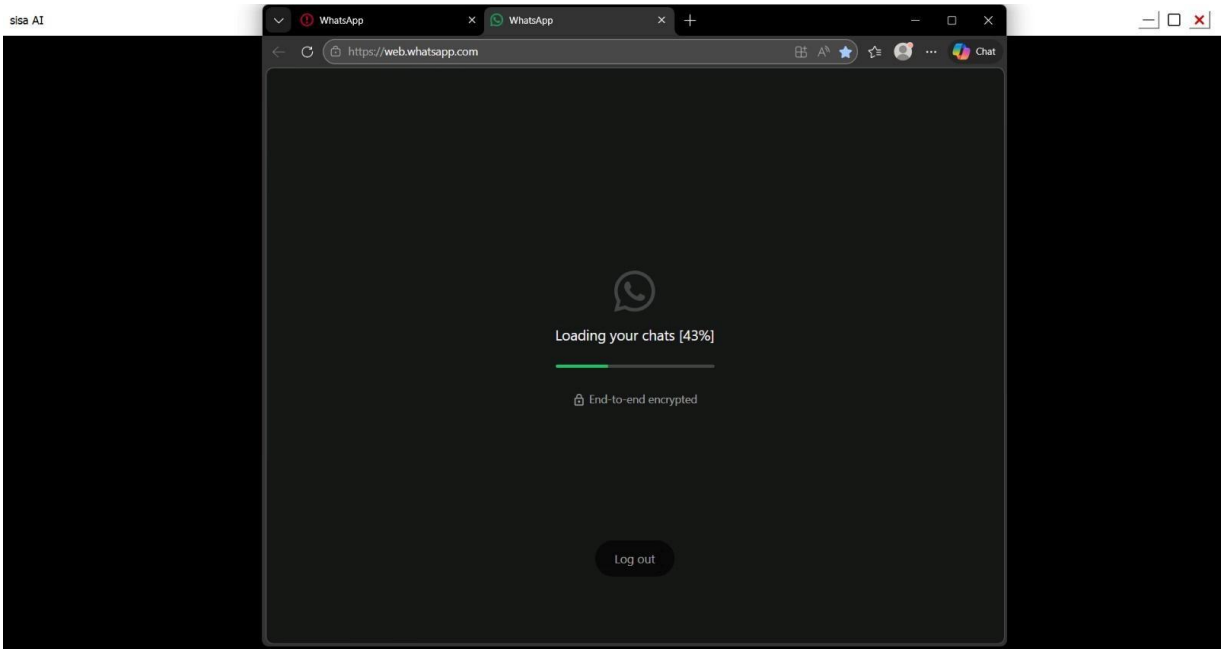
The backend acts as the engine of the assistant. Key coding components include:

- **Speech-to-Text (STT):** Implementation of the SpeechRecognition library to capture microphone input and convert it into a string format.
- **Text-to-Speech (TTS):** Integration of pyttsx3 for offline voice synthesis or gTTS for a more natural, cloud-based voice.
- **Command Router:** A "Logic Controller" that uses conditional mapping (If-Else or Match-Case) to execute tasks like opening applications, searching Wikipedia, or checking the system time.
- **Voice Biometrics:** Coding a specialized module that extracts the **MFCC (Mel-frequency cepstrum coefficients)** from the audio to verify if the speaker's vocal signature matches the authorized user.






sisa AI
Home
Chats
team ka captain bhi rahe hain aur unhone kai match jeetne mein mahatvapoorn bhoomika nibhai hai.
shagun : Can you tell me in english?
sisa: Virat Kohli is a renowned Indian cricketer, who is a brilliant batsman. He is considered one of the best batsmen in the world. He has been the captain of the Indian cricket team and has played a crucial role in winning many matches.
shagun : Can you open whatsapp?
shagun : Whatsapp mein?
shagun : Sabka project.
sisa : Everyone's project.
shagun : Shisha.
sisa : Glass.
shagun : Hey shisha will you tell me about virat kohli.
sisa : Unfortunately, I couldn't find any information on Virat Kohli as the search results were empty. If you could provide more context or details, I'd be happy to try and help you further.
shagun : He was cricketer.
sisa : Virat Kohli is a renowned Indian cricketer, widely regarded as one of the best batsmen in the world. He has played for the Indian national team and has been a key player in many of their victories. Kohli has also been the captain of the Indian team and has led them to several wins. He is known for his exceptional batting skills, his passion for the game, and his dedication to fitness and training.
Listening...


sisa AI
WhatsApp
WhatsApp
https://web.whatsapp.com
Chat
Loading your chats [43%]
End-to-end encrypted
Log out

5.2.2 Code Efficiency

Efficiency is critical for a voice assistant to feel "instant." We implemented several strategies to minimize latency and resource consumption:

1. **Non-Blocking I/O (Asynchronicity):** In Python, we used the threading or asyncio modules. This allows the assistant to "speak" while simultaneously preparing for the next command, preventing the application from "hanging" or becoming unresponsive during long tasks.
2. **Audio Buffer Optimization:** Instead of recording long audio files and then processing them, we implemented a **Silence Detection** threshold. The code stops recording as soon as the user finishes speaking (determined by a 1.5-second pause), which significantly reduces processing time.
3. **Frontend Caching:** In the React app, we cached repetitive responses (like "Hello" or "How can I help you?") to avoid unnecessary API calls to the server, reducing network load.
4. **Lightweight Libraries:** Instead of loading massive Machine Learning models (like heavy Transformer models) locally, we utilized existing, optimized libraries and APIs. This ensures the assistant can run on standard consumer laptops without requiring a dedicated GPU.
5. **Memory Management:** We ensured that the microphone resource is released (micclose()) immediately after a command is captured, preventing memory leaks and ensuring other applications can use the audio hardware.

CameraAnalysis.py

```
import google.generativeai as genai
import cv2
import numpy as np
import time
from Backend.TTS_B import speak  # Your custom TTS function
from PIL import Image
from dotenv import dotenv_values
import speech_recognition as sr  # For voice command detection
import json

# Configure Google Gemini AI API
env_vars = dotenv_values(".env")
GEMINI_API_KEY = env_vars.get("GeminiAPIKey")

genai.configure(api_key=GEMINI_API_KEY)

def analyze_screen(image):
    """Sends the captured frame to Google's Gemini AI for analysis."""
    model = genai.GenerativeModel("gemini-2.0-flash-exp")
    response = model.generate_content(["Describe in short what is happening in this frame:",
image])
    return response.text

def analyze_once():
    """Analyzes a single frame from the webcam and returns the analysis.
    This is a non-blocking version for integration with Jarvis.""" print("
```

```
        cap = cv2.VideoCapture(0)  # Start
webcam if not cap.isOpened():
        print("[+] Error: Could not open webcam")
        return None

    try:
        ret, frame = cap.read()
        if not ret:
            print("[+] Failed to capture frame")
            return None

        # Convert OpenCV frame to PIL Image
        image = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

        print("[ ] Analyzing webcam frame with Google AI...")
        analysis = analyze_screen(image)
        print("[ AI Analysis:", analysis)

        # Store the camera analysis in the chat log
        try:
            with open(r"Data\ChatLog.json", "r") as f:
                chat_log = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError):
            chat_log = []

        # Add the camera analysis to the chat log
        chat_log.append({
            "role": "system",
            "content": f"Camera Analysis: {analysis}"
        })

        # Save the updated chat log
        with open(r"Data\ChatLog.json", "w") as f:
            json.dump(chat_log, f, indent=4)

        return analysis

    except Exception as e:
        print(f"[ Error analyzing webcam: {str(e)}")
        return None
    finally:
        cap.release()
        cv2.destroyAllWindows()

def listen_for_command():
    """Listens for the user's voice command."""
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("\n[ ] Listening for command...")
        recognizer.adjust_for_ambient_noise(source)
```

```
        audio = recognizer.listen(source, timeout=5)  # Listen for 5 seconds max
            command = recognizer.recognize_google(audio).lower()
            print(f"[^] You said: {command}")
            return command
        except sr.UnknownValueError:
            print("[■] Couldn't understand the command.")
            return None
        except sr.RequestError:
            print("[■] Could not request results.")
            return None

def main():
    """Waits for the command to start and stop webcam analysis."""
    while True:
        command = listen_for_command()

        if command in ["start analysis", "analyze", "start"]:
            speak("Starting webcam analysis")
            print("\n[ ] Analyzing webcam feed with Google AI...")

            cap = cv2.VideoCapture(0)  # Start webcam
            if not cap.isOpened():
                print("[+] Error: Could not open webcam")
                speak("Error opening webcam")
                continue

            while True:
                ret, frame = cap.read()
                if not ret:
                    print("[+] Failed to capture frame")
                    break

                # Convert OpenCV frame to PIL Image
                image = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

                try:
                    analysis = analyze_screen(image)
                    print("[≤ AI Analysis:", analysis)
                    speak(analysis)
                except Exception as e:
                    print("[■] Error analyzing screen:", str(e))

                # Check if the user says "stop analysis"
                stop_command = listen_for_command()
                if stop_command in ["stop analysis", "stop"]:
                    speak("Stopping webcam analysis")
                    print("[●] Stopping analysis...")
                    break

                time.sleep(2)  # Adjust time interval
```

```
        cap.release()
        cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

Chatbot.py

```
from groq import Groq  # Importing the Groq library to use its API.
from json import load, dump  # Importing functions to read and write JSON files.
import datetime  # Importing the datetime module for real-time date and time information.
from dotenv import dotenv_values  # Importing dotenv_values to read environment
variables from a .env file.

# Load environment variables from the .env file.
env_vars = dotenv_values(".env")

# Retrieve specific environment variables for username, assistant name, and API key.
Username = env_vars.get("Username")
Assistantname = env_vars.get("Assistantname")
GroqAPIKey = env_vars.get("GroqAPIKey")
GroqModel = env_vars.get("GroqModel", "llama-3.3-70b-versatile")

# Initialize the Groq client using the provided API key.
client = Groq(api_key=GroqAPIKey)

# Initialize an empty list to store chat messages.
messages = []

# Define a system message that provides context to the AI chatbot about its role and
behavior.
System = f"""Hello, I am {Username}, You are a very accurate and advanced AI chatbot
named {Assistantname} which also has real-time up-to-date information from the internet.
*** Do not tell time until I ask, do not talk too much, just answer the question.***
*** Reply in only English, even if the question is in Hindi, reply in English.***
*** Do not provide notes in the output, just answer the question and never mention your
training data. ***
"""

# A list of system instructions for the chatbot.
SystemChatBot = [
    {"role": "system", "content": System}
]

try:
    with open(r"Data\ChatLog.json", "r") as f:
        messages = load(f)
except FileNotFoundError:
    with open(r"Data\ChatLog.json", "w") as f:
        dump([],f)

def RealtimeInformation():
    current_date_time = datetime.datetime.now()
```

```
    date = current_date_time.strftime("%d")


    month = current_date_time.strftime("%B")
    year = current_date_time.strftime("%Y")
    hour = current_date_time.strftime("%H")
    minute = current_date_time.strftime("%M")
    second = current_date_time.strftime("%S")

    data = f"Please use this real-time information if needed, \n"
    data += f"Date: {date} \nDate: {date}\nMonth: {month}\nYear: {year}\n"
    data += f"Time: {hour} hours:{minute} minutes:{second} seconds.\n"
    return data

def AnswerModifier(Answer):
    lines = Answer.split("\n")
    non_empty_lines = [line for line in lines if line.strip()]
    modified_answer = "\n".join(non_empty_lines)
    return modified_answer

def Chatbot(Query):
    """This function is used to answer the user's query."""

    try:
        with open(r"Data\ChatLog.json", "r") as f:
            messages = load(f)

        messages.append({"role": "user", "content": f"{Query}"})

        completion = client.chat.completions.create(
            model=GroqModel,
            messages=SystemChatBot + [{"role": "system", "content": RealtimeInformation()}]
+ messages,
            max_tokens=1024,
            temperature=0.7,
            top_p=1,
            stream=True,
            stop=None
        )

        Answer = ""

        for chunk in completion:
            if chunk.choices[0].delta.content:
                Answer += chunk.choices[0].delta.content

        Answer = Answer.replace("</s", "")

        messages.append({"role": "assistant", "content": Answer})

        with open(r"Data\ChatLog.json", "w") as f:
```

```
        return AnswerModifier(Answer=Answer)

    except Exception as e:
        print(f"Error: {e}")
        # Prevent infinite recursion if the API repeatedly fails.
        with open(r"Data\ChatLog.json", "w") as f:
            dump([], f, indent=4)
        return "Sorry, I couldn't process that request right now. Please try again later."


if __name__ == "__main__":
    while True:
        user_input = input("Enter Your Question: ")
        print(Chatbot(user_input))
```

ImageGeneration.py

```
 from huggingface_hub import InferenceClient
from PIL import Image
from dotenv import get_key
import os
from time import sleep

# Load API key from .env file
API_KEY = get_key(".env", "HuggingFaceAPIKey")

# Define Constants
FOLDER_PATH = r"Data"
IMAGE_STATUS_FILE = r"Frontend\Files\ImageGeneration.data"

# Initialize Hugging Face Inference Client
client = InferenceClient(
    provider="together",
    api_key=API_KEY
)

def open_images(prompt):
    """ Opens generated images based on the given prompt. """
    formatted_prompt = prompt.replace(" ", "_")
    files = [os.path.join(FOLDER_PATH, f"{formatted_prompt}{i}.jpg") for i in range(1,
5)]

    for image_path in files:
        try:
            with Image.open(image_path) as img:
                print(f"Opening Image: {image_path}")
                img.show()
                sleep(1)
        except IOError as e:
            print(f"Unable to open {image_path}: {e}")

def generate_images(prompt):
```

```
    formatted_prompt = prompt.replace(" ", "_")

    for i in range(1, 5):  # Generate 4 images
        try:
            image = client.text_to_image(
                prompt,
                model="stabilityai/stable-diffusion-xl-base-1.0"
            )

            image_path = os.path.join(FOLDER_PATH, f"{formatted_prompt}{i}.jpg")
            image.save(image_path)
            print(f"Image saved: {image_path}")

        except Exception as e:
            print(f"Error generating image {i}: {e}")

def generate_and_open_images(prompt):
    """ Runs the image generation and opens them. """
    generate_images(prompt)
    open_images(prompt)

if __name__ == "__main__":
    while True:
        try:
            if not os.path.exists(IMAGE_STATUS_FILE):
                print(f"File not found: {IMAGE_STATUS_FILE}")
                break

            with open(IMAGE_STATUS_FILE, "r") as f:
                data = f.read().strip()

            if not data:
                sleep(1)
                continue

            prompt, status = data.split(",")

            if status.strip().lower() == "true":
                print("Generating Images...")
                generate_and_open_images(prompt)

                with open(IMAGE_STATUS_FILE, "w") as f:
                    f.write("False, False")

                break  # Exit after generating images

            sleep(1)

        except Exception as e:
            print(f"Error:  {e}")
            sleep(1)
```

RealtimeSearchEngine.py

```
from googlesearch import search
from groq import Groq
from json import load, dump
import datetime
from dotenv import dotenv_values

env_vars = dotenv_values(".env")

Username = env_vars.get("Username")
Assistantname = env_vars.get("Assistantname")
GroqAPIKey = env_vars.get("GroqAPIKey")

client = Groq(api_key=GroqAPIKey)

System = f"""Hello, I am {Username}, You are a very accurate and advanced AI chatbot
named {Assistantname} which has real-time up-to-date information from the internet.
*** Provide Answers In a Professional Way, make sure to add full stops, commas, question
marks, and use proper grammar.***
*** Just answer the question from the provided data in a professional way. ***"""

try:
    with open(r"Data\ChatLog.json", "r") as f:
        messages = load(f)
except:
    with open(r"Data\ChatLog.json", "w") as f:
        dump([], f)

def GoogleSearch(query):
    """This function is used to search the internet for the given query."""

    results = list(search(query, advanced=True, num_results=5))
    Answer = f"The search results for {query} are: \n[start]\n"

    for i in results:
        Answer += f"Title: {i.title}\nDescription: {i.description}\n\n"

    Answer += "[end]"
    print(Answer)
    return Answer

def AnswerModifier(Answer):
    lines = Answer.split("\n")
    non_empty_lines = [line for line in lines if line.strip()]
    modified_answer = "\n".join(non_empty_lines)
    return modified_answer

SystemChatBot = [
    {"role": "system", "content": System},
    {"role": "user", "content": "Hi"},
    {"role": "assistant", "content": "Hello, how can I help you?"}
```

```
def Information():
    data = ""
    current_date_time = datetime.datetime.now()
    day = current_date_time.strftime("%A")
    date = current_date_time.strftime("%d")
    month = current_date_time.strftime("%B")
    year = current_date_time.strftime("%Y")
    hour = current_date_time.strftime("%H")
    minute = current_date_time.strftime("%M")
    second = current_date_time.strftime("%S")
    data += f"Please use this real-time information if needed:\n"
    data += f"Day: {day}\n"
    data += f"Date: {date}\n"
    data += f"Month: {month}\n"
    data += f"Year: {year}\n"
    data += f"Time: {hour} hours, {minute} minutes, {second} seconds.\n"
    return data

def RealtimeSearchEngine(prompt):
    global SystemChatBot, messages

    with open(r"Data\ChatLog.json", "r") as f:
        messages = load(f)

    messages.append({"role": "user", "content": f"{prompt}"})

    SystemChatBot.append({"role": "system", "content": GoogleSearch(prompt)})

    completion = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=SystemChatBot + [{"role": "system", "content": Information()}] +
messages,
        max_tokens=2048,
        temperature=0.7,
        top_p=1,
        stream=True,
        stop=None
    )

    Answer = ""


    for chunk in completion:
        if chunk.choices[0].delta.content:
            Answer += chunk.choices[0].delta.content

    Answer = Answer.replace("</s", "")

    messages.append({"role": "assistant", "content": Answer})

    with open(r"Data\ChatLog.json", "w") as f:
```

```
    SystemChatBot.pop()
    return AnswerModifier(Answer=Answer)

if __name__ == "__main__":
    while True:
        prompt = input("Enter your query: ")
        print(RealtimeSearchEngine(prompt))
```

screen_analysis.py

```
import google.generativeai as genai
import cv2
import numpy as np
import mss
import time
import speech_recognition as sr
from PIL import Image
from Backend.TTS_B import speak  # Your custom TTS function
from dotenv import dotenv_values
import json

# Configure Google Gemini AI API
env_vars = dotenv_values(".env")
GEMINI_API_KEY = env_vars.get("GeminiAPIKey")

# Configure Google Gemini AI API
genai.configure(api_key=GEMINI_API_KEY)

def capture_screen():
    """Captures a frame from the screen using MSS."""
    with mss.mss() as sct:
        monitor = sct.monitors[1]  # Capture the primary screen
        screenshot = sct.grab(monitor)

        # Convert the screenshot to a numpy array (for OpenCV processing)
        img = np.array(screenshot)
        img = cv2.cvtColor(img, cv2.COLOR_BGRA2RGB)  # Convert BGRA to RGB

        return Image.fromarray(img)  # Convert to PIL Image

def analyze_screen(image):
    """Sends the screen frame to Google's Gemini AI for analysis."""
    model = genai.GenerativeModel("gemini-2.0-flash-exp")
    response = model.generate_content(["Describe in short what is happening in this screen
frame:", image])
    return response.text

def listen_for_command():
    """Listens for a voice command to activate screen analysis."""
    recognizer = sr.Recognizer()
```

```
    with sr.Microphone() as source:
        print("\n Say 'Jarvis, analyze screen' to trigger AI... (Waiting for command)")

        # Adjust for ambient noise to avoid incorrect recognition
        recognizer.adjust_for_ambient_noise(source, duration=1)

        try:
            # Listen to the command and set a timeout for waiting
            audio = recognizer.listen(source, timeout=5)

            print("[ ] Listening...")
            command = recognizer.recognize_google(audio).lower()  # Convert speech to text
            print(f"[ Command recognized: {command}")

            return command
        except sr.UnknownValueError:
            print("[ Could not understand the audio")
            return None
        except sr.RequestError:
            print("[ Voice recognition service unavailable")
            return None
        except Exception as e:
            print(f"[ Error during recognition: {e}")
            return None

def analyze_once():
    """Analyzes the screen once and returns to the main program.
    This is a non-blocking version for integration with Jarvis."""
    print("\n[ ] Capturing screen...")
    image = capture_screen()

    print("[ ] Analyzing screen with Google AI...")
    try:
        analysis = analyze_screen(image)
        print("[ ] AI Analysis:", analysis)

        # Store the screen analysis in the chat log
        try:
            with open(r"Data\ChatLog.json", "r") as f:
                chat_log = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError):
            chat_log = []

        # Add the screen analysis to the chat log
        chat_log.append({
            "role": "system",
            "content": f"Screen Analysis: {analysis}"
        })

        # Save the updated chat log
```

```
            json.dump(chat_log, f, indent=4)

        return analysis  # Return the analysis instead of speaking it
    except Exception as e:
        print(f"[ Error analyzing screen: {str(e)}")
        return None

def main():
    """Waits for a command to analyze the screen."""
    while True:
        command = listen_for_command()

        if command and "analyse screen" in command:
            print("\n[ ] Capturing screen...")
            image = capture_screen()

            print("[ ] Analyzing screen with Google AI...")
            try:
                analysis = analyze_screen(image)
                print("[ AI Analysis:", analysis)

                # Store the screen analysis in the chat log
                try:
                    with open(r"Data\ChatLog.json", "r") as f:
                        chat_log = json.load(f)
                except (FileNotFoundError, json.JSONDecodeError):
                    chat_log = []

                # Add the screen analysis to the chat log
                chat_log.append({
                    "role": "system",
                    "content": f"Screen Analysis: {analysis}"
                })

                # Save the updated chat log
                with open(r"Data\ChatLog.json", "w") as f:
                    json.dump(chat_log, f, indent=4)

                # Speak the AI-generated analysis
                speak(analysis)
            except Exception as e:
                print("[ Error analyzing screen:", str(e))

        time.sleep(1)  # Small delay to avoid excessive processing

if __name__ == "__main__":
    main()
```

TextToSpeech.py

```
import pygame
```

```
import os
from dotenv import dotenv_values
import edge_tts

env_vars = dotenv_values(".env")
AssistantVoice = env_vars.get("AssistantVoice")

async def TextToAudioFile(text) -> None:
    file_path = r"Data\speech.mp3"

    if os.path.exists(file_path):
        os.remove(file_path)

    communicate = edge_tts.Communicate(text, AssistantVoice, pitch='+5Hz', rate='+13%')
    await communicate.save(r'Data\speech.mp3')

def TTS(Text, func=lambda r=None: True):
    while True:
        try:
            asyncio.run(TextToAudioFile(Text))
            pygame.mixer.init()
            pygame.mixer.music.load(r'Data\speech.mp3')
            pygame.mixer.music.play()

            while pygame.mixer.music.get_busy():
                if func() == False:
                    break
                pygame.time.Clock().tick(10)

            return True

        except Exception as e:
            print(f"Error in TTS: {e}")

        finally:
            try:

                func(False)
                pygame.mixer.music.stop()
                pygame.mixer.quit()

            except Exception as e:
                print(f"Error in finally block: {e}")

def TextToSpeech(Text, func=lambda r=None: True):
    Data = str(Text).split(".")

    responses = [
        "The rest of the result has been printed to the chat screen, kindly check it out sir.",
        "The rest of the text is now on the chat screen, sir, please check it.",
        "You can see the rest of the text on the chat screen, sir.",
```

```
        "The rest of the answer is now on the chat screen, sir.",
        "Sir, please look at the chat screen, the rest of the answer is there.",
        "You'll find the complete answer on the chat screen, sir.",
        "The next part of the text is on the chat screen, sir.",
        "Sir, please check the chat screen for more information.",
        "There's more text on the chat screen for you, sir.",
        "Sir, take a look at the chat screen for additional text.",
        "You'll find more to read on the chat screen, sir.",
        "Sir, check the chat screen for the rest of the text.",
        "The chat screen has the rest of the text, sir.",
        "There's more to see on the chat screen, sir, please look.",
        "Sir, the chat screen holds the continuation of the text.",
        "You'll find the complete answer on the chat screen, kindly check it out sir.",
        "Please review the chat screen for the rest of the text, sir.",
        "Sir, look at the chat screen for the complete answer."
    ]

    if len(Data) > 4 and len(Text) >= 250:
        TTS(" ".join(Text.split(".")[0:2]) + ". " + random.choice(responses), func)

    else:
        TTS(Text, func)

if __name__ == "__main__":
    while True:
        TextToSpeech(input("Enter the text: "))
```

Data

ChatLog.json

```
[
    {
        "role": "user",
        "content": "Can you play song?"
    },
    {
        "role": "assistant",
        "content": "I can provide you with song information, lyrics, or recommendations, but I'm not capable of playing audio."
    },
    {
        "role": "user",
        "content": "Nazar lag jayegi."
    },
    {
        "role": "assistant",
        "content": "It seems like you're referencing a Hindi phrase, but I'll respond in English as per our conversation guidelines. The phrase \"nazar lag jayegi\" is often used to ward off evil spirits or bad luck."
    },
```

```
      "role": "user",
      "content": "Security security."
    },
    {
      "role": "assistant",
      "content": "It seems like you're looking for information on security measures or
protocols. What specific aspect of security would you like to know about?"
    },
    {
      "role": "user",
      "content": "Tu mera ko savere ka tu mere suraj."
    },
    {
      "role": "assistant",
      "content": "It appears to be a poetic phrase, likely from a song or a romantic
expression. The translation would be \"You are my morning, you are my sun.\""
    },
    {
      "role": "user",
      "content": "Hello can you search?"
    },
    {
      "role": "assistant",
      "content": "Yes, I can search for information on various topics. What would you like
me to search for?"
    },
    {
      "role": "user",
      "content": "What's my name?"
    },
    {
      "role": "assistant",
      "content": "Your name is Pradeep."
    },
    {
      "role": "user",
      "content": "Hello."
    },
    {
      "role": "assistant",
      "content": "Hello Pradeep."
    },
    {
      "role": "user",
      "content": "Can you search my instagram id?"
    },
    {
      "role": "assistant",
      "content": "I don't have direct access to external accounts or personal information. I
can provide general information or help with a topic, but I won't be able to search for your
specific Instagram ID."
```

```
        "role": "user",
        "content": "Ab jaise main kiska kiska banaa jo jiska private nahin ho."
    },
    {
        "role": "assistant",
        "content": "It seems like you're expressing a sentiment about being open and
accessible to others, rather than being private. The phrase can be roughly translated to
\"Now, as I belong to everyone, whoever's I am, I am not private.\""
    },
    {
        "role": "user",
        "content": "Shera hai lekin tere mein maine dekha."
    },
    {
        "role": "assistant",
        "content": "It appears to be a poetic phrase, likely from a song or a romantic
expression. The translation would be \"You are a lion, but I have seen a gentle side in
you.\""
    },
    {
        "role": "user",
        "content": "Don't call tanvi shekhar dot 13."
    },
    {
        "role": "assistant",
        "content": "I won't call anyone. I'm just a text-based AI assistant, I don't have the
capability to make phone calls."
    },
    {
        "role": "user",
        "content": "Net slow chal raha hai."
    },
    {
        "role": "assistant",
        "content": "It seems like you're experiencing slow internet speeds."
    },
    {
        "role": "user",
        "content": "Nahin bhai usne."
    },
    {
        "role": "assistant",
        "content": "It seems like you're referring to someone else experiencing an issue, but
the context is unclear. Can you provide more information?"
    }
]
```

Voice.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<body>
    <button id="start" onclick="startRecognition()">Start Recognition</button>
    <button id="end" onclick="stopRecognition()">Stop Recognition</button>
    <p id="output"></p>
    <script>
        const output = document.getElementById('output');
        let recognition;

        function startRecognition() {
            recognition = new webkitSpeechRecognition() || new SpeechRecognition();
            recognition.lang = 'en';
            recognition.continuous = true;

            recognition.onresult = function(event) {
                const transcript = event.results[event.results.length - 1][0].transcript;
                output.textContent += transcript;
            };

            recognition.onend = function() {
                recognition.start();
            };
            recognition.start();
        }

        function stopRecognition() {
            recognition.stop();
            output.innerHTML = "";
        }
    </script>
</body>
</html>
```

5.3 Testing Approach

The testing phase was designed to ensure that the AI Voice Assistant is reliable, responsive, and secure. A **Top-Down Testing Strategy** was adopted, starting from individual code functions and moving toward the complete integration of the web interface with the voice recognition engine. The primary goal was to validate that the system executes tasks based on "Speech-to-Do" commands while maintaining the integrity of the authorized user's vocal signature.

5.3.1 Unit Testing

Unit testing involved isolating each modular component of the Python backend and React frontend to verify its logic independently.

- **Speech-to-Text (STT) Validation:** Testing the SpeechRecognition module with pre-recorded audio clips to ensure 95% accuracy in transcription

```
from selenium import webdriver
```

```
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager
from dotenv import dotenv_values
import os
import mtranslate as mt


env_vars = dotenv_values(".env")

InputLanguage = env_vars.get("InputLanguage")

HtmlCode = '''<!DOCTYPE html>
<html lang="en">
<head>
    <title>Speech Recognition</title>
</head>
<body>
    <button id="start" onclick="startRecognition()">Start Recognition</button>
    <button id="end" onclick="stopRecognition()">Stop Recognition</button>
    <p id="output"></p>
    <script>
        const output = document.getElementById('output');
        let recognition;

        function startRecognition() {
            recognition = new webkitSpeechRecognition() || new SpeechRecognition();
            recognition.lang = '';
            recognition.continuous = true;

            recognition.onresult = function(event) {
                const transcript = event.results[event.results.length - 1][0].transcript;
                output.textContent += transcript;
            };

            recognition.onend = function() {
                recognition.start();
            };
            recognition.start();
        }

        function stopRecognition() {
            recognition.stop();
            output.innerHTML = "";
        }
    </script>
</body>
</html>'''

HtmlCode = str(HtmlCode).replace("recognition.lang = '';", f"recognition.lang =
'{InputLanguage}';")
```

```
current_dir = os.getcwd()

Link = f"{current_dir}/Data/Voice.html"

chrome_options = Options()
user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/89.0.142.86 Safari/537.36"
chrome_options.add_argument(f"user-agent={user_agent}")
chrome_options.add_argument("--use-fake-ui-for-media-stream")
chrome_options.add_argument("--use-fake-device-for-media-stream")
chrome_options.add_argument("--headless=new")

service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service, options=chrome_options)

TempDirPath = rf"{current_dir}/Frontend/Files"

def SetAssistantStatus(Status):
    with open(rf'{TempDirPath}/Status.data', "w", encoding="utf-8") as file:
        file.write(Status)

def QueryModifier(Query):
    new_query = Query.lower().strip()
    query_words = new_query.split()
    question_words = ["how", "what", "why", "when", "where", "who", "which", "whom",
"whose", "can you", "what's", "where's", "how's"]

    if any(word + "" in new_query for word in question_words):
        if query_words[-1][-1] in ['.', '?', '!']:
            new_query = new_query[:-1] + "?"
        else:
            new_query += "?"
    else:
        if query_words[-1][-1] in ['.', '?', '!']:
            new_query = new_query[:-1] + "."
        else:
            new_query += "."

    return new_query.capitalize()
def UniversalTranslator(Text):
    english_translation = mt.translate(Text, "en", "auto")
    return english_translation.capitalize()

def SpeechRecognition():
    driver.get("file:///" + Link)
    driver.find_element(by=By.ID, value="start").click()

    while True:
        try:
```

```
            if Text:
                driver.find_element(by=By.ID, value="end").click()

                if InputLanguage.lower() == "en" or "en" in InputLanguage.lower():
                    return QueryModifier(Text)
                else:
                    SetAssistantStatus("Translating...")
                    return QueryModifier(UniversalTranslator(Text))

        except Exception as e:
            pass

if __name__ == "__main__":
    while True:
        Text = SpeechRecognition()
        print(Text)
```

.

- **Command Logic:** Testing the "Task Execution" functions (e.g., open_google(), get_time()) by passing string inputs manually to see if the correct system action is triggered.

```
from AppOpener import close, open as appopen
from webbrowser import open as webopen
from pywhatkit import search, playonyt
from dotenv import dotenv_values
from bs4 import BeautifulSoup
import requests
from rich import print
from groq import Groq
import os
import webbrowser
import subprocess
import keyboard
import asyncio
from urllib.parse import quote

env_vars = dotenv_values(".env")
GroqAPIKey = env_vars.get("GroqAPIKey")

classes = ["zCubwf", "hgKElc", "LTKOO sY7ric", "z0LcW", "gsrt vk_bk FzvWSb
YwPhnf", "pclqee", "tw-Data-text tw-text-small tw-ta",
        "IZ6rdc", "O5uR6d LTKOO", "vlzY6d", "webanswers-
webanswers_table__webanswers-table", "dDoNo ikb4Bb gsrt", "sXLaOe",
        "LWkfKe", "VQF4g", "qv3Wpe", "kno-rdesc", "SPZz6b"]

useragent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/100.0.4896.75 Safari/537.36"

client = Groq(api_key=GroqAPIKey)
```

```
    "I'm here to assist you with anything you need. Feel free to ask anytime!",
    "Your queries are always welcome, and I'm happy to help you further."
]

# List to store chatbot messages
messages = []

# System message providing context to the chatbot
SystemChatBot = [{"role": "system", "content": f"Hello, I am {os.environ['Username']},
You're a content writer. You have to write content like letter."}]
# Function to perform a Google search
def GoogleSearch(topic):
    search(topic) # Assuming a search function to perform Google search
    return True # Indicate success

# Function to generate content using AI and save it to a file.
def Content(Topic):

    # Nested function to open a file in Notepad.
    def OpenNotepad(File):
        default_text_editor = 'notepad.exe' # Default text editor.
        subprocess.Popen([default_text_editor, File]) # Open the file in Notepad.

    # Nested function to generate content using the AI chatbot.
    def ContentWriterAI(prompt):
        messages.append({"role": "user", "content": f"{prompt}"}) # Add the user's prompt to
messages.

        completion = client.chat.completions.create(
            model="llama-3.3-70b-versatile", # Specify the AI model.
            messages=SystemChatBot + messages, # Include system instructions and chat
history.
            max_tokens=2048, # Limit on the number of tokens in the output.
            temperature=0.7, # Adjust the randomness of the output.
            top_p=1,
            stream=True,
            stop=None
        )

        Answer = ""

        for chunk in completion:
            if chunk.choices[0].delta.content:
                Answer += chunk.choices[0].delta.content

        Answer = Answer.replace("</s>", "")
        messages.append({"role": "assistant", "content": Answer})
        return Answer

    Topic: str = Topic.replace("Content ", "")
    ContentByAI = ContentWriterAI(Topic)
```

```
        file.write(ContentByAI)
        file.close()

    OpenNotepad(rf"Data\{Topic.lower().replace(' ',"_")}.txt")
    return True

def YouTubeSearch(Topic):
    Url4Search = f"https://www.youtube.com/results?search_query={Topic}"  # Construct
the YouTube search URL.
    webbrowser.open(Url4Search)  # Open the search URL in a web browser.
    return True  # Indicate success.

# Function to play a video on YouTube.
def PlayYoutube(query):
    playonyt(query)  # Use pywhatkit's playonyt function to play the video.
    return True  # Indicate success.


APP_WEBSITES = {
    # 🎬 Streaming Services
    "netflix":  "https://www.netflix.com",
    "netflix.": "https://www.netflix.com",
    "amazon prime":  "https://www.primevideo.com",
    "amazon prime.": "https://www.primevideo.com",
    "hotstar": "https://www.hotstar.com",
    "hotstar.": "https://www.hotstar.com",
    "hulu": "https://www.hulu.com",
    "hulu.": "https://www.hulu.com",
    "disney+":  "https://www.disneyplus.com",
    "disney+.": "https://www.disneyplus.com",
    "apple tv": "https://tv.apple.com",
    "hbo max": "https://www.hbomax.com",
    "hbo max.": "https://www.hbomax.com",
    "youtube": "https://www.youtube.com",
    "youtube.": "https://www.youtube.com",

    # 🎵 Music & Audio Streaming
    "spotify": "https://www.spotify.com",
    "spotify.": "https://www.spotify.com",
    "apple music": "https://music.apple.com",
    "apple music.": "https://music.apple.com",
    "soundcloud": "https://soundcloud.com",
    "soundcloud.": "https://soundcloud.com",
    "gaana": "https://gaana.com",
    "gaana.": "https://gaana.com",
    "wynk music":  "https://wynk.in/music",
    "wynk music.":  "https://wynk.in/music",
    "jiosaavn":  "https://www.jiosaavn.com",
    "jiosaavn.": "https://www.jiosaavn.com",

    # 💬 Social Media & Communication
```

"facebook.": "https://www.facebook.com",
"instagram": "https://www.instagram.com",
"instagram.": "https://www.instagram.com",
"twitter": "https://twitter.com",
"twitter.": "https://twitter.com",
"linkedin": "https://www.linkedin.com",
"linkedin.": "https://www.linkedin.com",
"snapchat": "https://www.snapchat.com",
"snapchat.": "https://www.snapchat.com",
"reddit": "https://www.reddit.com",
"reddit.": "https://www.reddit.com",
"telegram": "https://web.telegram.org",
"telegram.": "https://web.telegram.org",
"whatsapp": "https://web.whatsapp.com",
"whatsapp.": "https://web.whatsapp.com",
"discord": "https://discord.com",
"discord.": "https://discord.com",
"messenger": "https://www.messenger.com",
"messenger.": "https://www.messenger.com",
"tiktok": "https://www.tiktok.com",
"tiktok.": "https://www.tiktok.com",

Search Engines
"google": "https://www.google.com",
"google.": "https://www.google.com",
"bing": "https://www.bing.com",
"bing.": "https://www.bing.com",
"duckduckgo": "https://www.duckduckgo.com",
"duckduckgo.": "https://www.duckduckgo.com",
"yahoo": "https://www.yahoo.com",
"yahoo.": "https://www.yahoo.com",

Developer & Coding Platforms
"github": "https://github.com",
"github.": "https://github.com",
"gitlab": "https://gitlab.com",
"gitlab.": "https://gitlab.com",
"stackoverflow": "https://stackoverflow.com",
"stackoverflow.": "https://stackoverflow.com",
"chatgpt": "https://chat.openai.com",
"chatgpt.": "https://chat.openai.com",
"huggingface": "https://huggingface.co",
"huggingface.": "https://huggingface.co",
"kaggle": "https://www.kaggle.com",
"kaggle.": "https://www.kaggle.com",
"w3schools": "https://www.w3schools.com",
"w3schools.": "https://www.w3schools.com",
"leetcode": "https://leetcode.com",
"leetcode.": "https://leetcode.com",
"codeforces": "https://codeforces.com",
"codeforces.": "https://codeforces.com",

Productivity & Work Tools
"zoom": "https://zoom.us",
"zoom.": "https://zoom.us",
"microsoft teams": "https://teams.microsoft.com",
"microsoft teams.": "https://teams.microsoft.com",
"slack": "https://slack.com",
"slack.": "https://slack.com",
"notion": "https://www.notion.so",
"notion.": "https://www.notion.so",
"evernote": "https://www.evernote.com",
"evernote.": "https://www.evernote.com",
"google drive": "https://drive.google.com",
"google drive.": "https://drive.google.com",
"onedrive": "https://onedrive.live.com",
"onedrive.": "https://onedrive.live.com",
"dropbox": "https://www.dropbox.com",
"dropbox.": "https://www.dropbox.com",
"canva": "https://www.canva.com",
"canva.": "https://www.canva.com",
"figma": "https://www.figma.com",
"figma.": "https://www.figma.com",
"trello": "https://trello.com",
"trello.": "https://trello.com",

Education & Learning
"coursera": "https://www.coursera.org",
"coursera.": "https://www.coursera.org",
"udemy": "https://www.udemy.com",
"udemy.": "https://www.udemy.com",
"udemi": "https://www.udemy.com",
"udemi.": "https://www.udemy.com",
"edx": "https://www.edx.org",
"edx.": "https://www.edx.org",
"khan academy": "https://www.khanacademy.org",
"khan academy.": "https://www.khanacademy.org",
"brilliant": "https://www.brilliant.org",
"brilliant.": "https://www.brilliant.org",
"chegg": "https://www.chegg.com",
"chegg.": "https://www.chegg.com",
"wolfram alpha": "https://www.wolframalpha.com",
"wolfram alpha.": "https://www.wolframalpha.com",
"nptel": "https://nptel.ac.in",
"nptel.": "https://nptel.ac.in",
"byju's": "https://byjus.com",
"byju's.": "https://byjus.com",

Shopping & E-commerce
"amazon": "https://www.amazon.com",
"amazon.": "https://www.amazon.com",
"flipkart": "https://www.flipkart.com",
"flipkart.": "https://www.flipkart.com",

"snapdeal": "https://www.snapdeal.com",
"snapdeal.": "https://www.snapdeal.com",
"shopify": "https://www.shopify.com",
"shopify.": "https://www.shopify.com",
"myntra": "https://www.myntra.com",
"ajio": "https://www.ajio.com",

🏦 Banking & Finance
"paypal": "https://www.paypal.com",
"paypal.": "https://www.paypal.com",
"stripe": "https://stripe.com",
"stripe.": "https://stripe.com",
"paytm": "https://paytm.com",
"paytm.": "https://paytm.com",
"phonepe": "https://www.phonepe.com",
"phonepe.": "https://www.phonepe.com",
"google pay": "https://pay.google.com",
"google pay.": "https://pay.google.com",
"razorpay": "https://razorpay.com",
"razorpay.": "https://razorpay.com",
"upstox": "https://upstox.com",
"upstox.": "https://upstox.com",
"zerodha": "https://zerodha.com",
"zerodha.": "https://zerodha.com",
"groww": "https://groww.in",
"groww.": "https://groww.in",

🎮 Gaming & Esports
"twitch": "https://www.twitch.tv",
"twitch.": "https://www.twitch.tv",
"steam": "https://store.steampowered.com",
"steam.": "https://store.steampowered.com",
"epic games": "https://www.epicgames.com",
"epic games.": "https://www.epicgames.com",
"roblox": "https://www.roblox.com",
"roblox.": "https://www.roblox.com",
"fortnite": "https://www.fortnite.com",
"fortnite.": "https://www.fortnite.com",
"valorant": "https://playvalorant.com",
"valorant.": "https://playvalorant.com",
"pubg": "https://www.pubg.com",
"pubg.": "https://www.pubg.com",

📰 News & Information
"bbc": "https://www.bbc.com",
"bbc.": "https://www.bbc.com",
"cnn": "https://www.cnn.com",
"cnn.": "https://www.cnn.com",
"nytimes": "https://www.nytimes.com",
"nytimes.": "https://www.nytimes.com",
"the guardian": "https://www.theguardian.com",

```
    "hindustan times.": "https://www.hindustantimes.com",
    "times of india": "https://timesofindia.indiatimes.com",
    "times of india.": "https://timesofindia.indiatimes.com",

    # 💤 Miscellaneous
    "speedtest": "https://www.speedtest.net",
    "speedtest.": "https://www.speedtest.net",
    "weather": "https://www.weather.com",
    "weather.": "https://www.weather.com",
    "translate": "https://translate.google.com",
    "translate.": "https://translate.google.com",
    "wikipedia": "https://www.wikipedia.org",
    "wikipedia.": "https://www.wikipedia.org",
    "quora": "https://www.quora.com",
    "quora.": "https://www.quora.com",
    "medium": "https://medium.com",
    "medium.": "https://medium.com",

}



def OpenApp(app, sess=requests.session()):
    app_lower = app.lower()

    # Try opening the installed app
    try:
        print(f"[cyan]Trying to open {app}...[/cyan]")
        appopen(app, match_closest=True, output=True, throw_error=True)
        return True

    except Exception:
        print(f"[yellow]Could not open {app} locally. Opening web version...[/yellow]")

        # Check predefined websites
        if app_lower in APP_WEBSITES:
            print(f"[green]Opening official website: {APP_WEBSITES[app_lower]}[/green]")
            webopen(APP_WEBSITES[app_lower])
            return True

        # If not found, fallback to Google search
        def search_google(query):
            url = f"https://www.google.com/search?q={query} official site"
            headers = {"User-Agent": useragent}
            response = sess.get(url, headers=headers)
            return response.text if response.status_code == 200 else None

        def extract_links(html):
            if not html:
                return []
            soup = BeautifulSoup(html, "html.parser")
```

```
        # Perform Google search
        html = search_google(app)
        links = extract_links(html)

        if links:
            print(f"[green]Opening official website: {links[0]}[/green]")
            webopen(links[0])  # Open first valid link
        else:
            print(f"[red]Could not find an official site for {app}.[/red]")

        return True


def SendWhatsAppMessage(message: str, phone: str = None):
    """Open WhatsApp Web with a prefilled message (optionally to a specific phone
number)."""
    base_url = "https://web.whatsapp.com/send?"
    params = []
    if phone:
        params.append(f"phone={phone}")
    if message:
        params.append(f"text={quote(message)}")
    url = base_url + "&".join(params)
    print(f"[green]Opening WhatsApp Web with message: {message}[/green]")
    webopen(url)
    return True

def CloseApp(app):

    if "chrome" in app:
        pass
    else:
        try:
            close(app, match_closest=True, output=True, throw_error=True)
            return True
        except:
            return False

def System(command):

    def mute():
        keyboard.press_and_release("volume mute")

    def unmute():
        keyboard.press_and_release("volume mute")

    def volume_up():
        keyboard.press_and_release("volume up")

    def volume_down():
        keyboard.press_and_release("volume down")
```

```
    if command == "mute":
        mute()
    elif command == "unmute":
        unmute()
    elif command == "volume up":
        volume_up()
    elif command == "volume down":
        volume_down()

    return True

async def TranslateAndExecute(commands: list[str]):

    funcs = []

    for command in commands:

        if command.startswith("open "):

            if "open it" in command:
                pass

            if "open file" == command:
                pass

            else:
                fun = asyncio.to_thread(OpenApp, command.removeprefix("open "))
                funcs.append(fun)

        elif command.startswith("general "):
            pass

        elif command.startswith("realtime "):
            pass

        elif command.startswith("send whatsapp "):
            message = command.removeprefix("send whatsapp ").strip()
            if message:
                fun = asyncio.to_thread(SendWhatsAppMessage, message)
                funcs.append(fun)

        elif command.startswith("close "):
            fun = asyncio.to_thread(CloseApp, command.removeprefix("close "))
            funcs.append(fun)

        elif command.startswith("play "):
            fun = asyncio.to_thread(PlayYoutube, command.removeprefix("play "))
            funcs.append(fun)

        elif command.startswith("content "):
            fun = asyncio.to_thread(Content, command.removeprefix("content "))
```

```
        elif command.startswith("youtube search "):
            fun = asyncio.to_thread(YouTubeSearch, command.removeprefix("youtube search
"))
            funcs.append(fun)

        elif command.startswith("system "):
            fun = asyncio.to_thread(System, command.removeprefix("system "))
            funcs.append(fun)

        elif command.startswith("google search "):
            fun = asyncio.to_thread(GoogleSearch, command.removeprefix("google search "))
            funcs.append(fun)

        else:
            print(f"No Function Found. For {command}")


    results = await asyncio.gather(*funcs)

    for result in results:
        if isinstance(result, str):
            yield result
        else:
            yield result


async def Automation(commands: list[str]):

    async for result in TranslateAndExecute(commands):
        pass

    return True

# if __name__ == "__main__":
#     asyncio.run(Automation(["open netflix","open spotify", "open zoom", "open
whatsapp", "open telegram"]))
```

- **UI Component Testing:** Verifying that the React "Microphone" button changes state correctly (from Idle to Listening) when clicked.

```
import cohere
from rich import print
from dotenv import dotenv_values

env_vars = dotenv_values(".env")

CohereAPIKey = env_vars.get("CohereAPIKey")
CohereModel = env_vars.get("CohereModel", "command-xlarge-nightly")

co = cohere.Client(api_key=CohereAPIKey)

funcs = [
    "exit", "general", "realtime", "open", "close", "play",
    "generate image", "system", "content", "google search",
    "youtube search", "reminder"
]

messages = []


preamble = """
You are a very accurate Decision-Making Model, which decides what kind of a query is given to you.
You will decide whether a query is a 'general' query, a 'realtime' query, or is asking to perform any task or automation like 'open facebook, instagram', 'can you write a application and open it in notepad'
*** Do not answer any query, just decide what kind of query is given to you. ***
-> Respond with 'general ( query )' if a query can be answered by a llm model (conversational ai chatbot) and doesn't require any up to date information like if the query is 'who was akbar?' respond with 'general who was akbar?', if the query is 'how can i study more effectively?' respond with 'general how can i study more effectively?', if the query is 'can you help me with this math problem?' respond with 'general can you help me with this math problem?', if the query is 'Thanks, i really liked it.' respond with 'general thanks, i really liked it.' , if the query is 'what is python programming language?' respond with 'general what is python programming language?', etc. Respond with 'general (query)' if a query doesn't have a proper noun or is incomplete like if the query is 'who is he?' respond with 'general who is he?', if the query is 'what's his networth?' respond with 'general what's his networth?', if the query is 'tell me more about him.' respond with 'general tell me more about him.', and so on even if it require up-to-date information to answer. Respond with 'general (query)' if the query is asking about time, day, date, month, year, etc like if the query is 'what's the time?' respond with 'general what's the time?'.
-> Respond with 'realtime ( query )' if a query can not be answered by a llm model (because they don't have realtime data) and requires up to date information like if the query is 'who is indian prime minister' respond with 'realtime who is indian prime minister', if the query is 'tell me about facebook's recent update.' respond with 'realtime tell me about facebook's recent update.', if the query is 'tell me news about coronavirus.' respond with 'realtime tell me news about coronavirus.', etc and if the query is asking about any individual or thing like if the query is 'who is akshay kumar' respond with 'realtime who is akshay kumar', if the query is 'what is today's news?' respond with 'realtime what is today's news?', if the query is 'what is today's
```

-> Respond with 'open (application name or website name)' if a query is asking to open any application like 'open facebook', 'open telegram', etc. but if the query is asking to open multiple applications, respond with 'open 1st application name, open 2nd application name' and so on.
-> Respond with 'close (application name)' if a query is asking to close any application like 'close notepad', 'close facebook', etc. but if the query is asking to close multiple applications or websites, respond with 'close 1st application name, close 2nd application name' and so on.
-> Respond with 'play (song name)' if a query is asking to play any song like 'play afsanay by ys', 'play let her go', etc. but if the query is asking to play multiple songs, respond with 'play 1st song name, play 2nd song name' and so on.
-> Respond with 'generate image (image prompt)' if a query is requesting to generate a image with given prompt like 'generate image of a lion', 'generate image of a cat', etc. but if the query is asking to generate multiple images, respond with 'generate image 1st image prompt, generate image 2nd image prompt' and so on.
-> Respond with 'reminder (datetime with message)' if a query is requesting to set a reminder like 'set a reminder at 9:00pm on 25th june for my business meeting.' respond with 'reminder 9:00pm 25th june business meeting'.
-> Respond with 'system (task name)' if a query is asking to mute, unmute, volume up, volume down , etc. but if the query is asking to do multiple tasks, respond with 'system 1st task, system 2nd task', etc.
-> Respond with 'content (topic)' if a query is asking to write any type of content like application, codes, emails or anything else about a specific topic but if the query is asking to write multiple types of content, respond with 'content 1st topic, content 2nd topic' and so on.
-> Respond with 'google search (topic)' if a query is asking to search a specific topic on google but if the query is asking to search multiple topics on google, respond with 'google search 1st topic, google search 2nd topic' and so on.
-> Respond with 'youtube search (topic)' if a query is asking to search a specific topic on youtube but if the query is asking to search multiple topics on youtube, respond with 'youtube search 1st topic, youtube search 2nd topic' and so on.
*** If the query is asking to perform multiple tasks like 'open facebook, telegram and close whatsapp' respond with 'open facebook, open telegram, close whatsapp' ***
*** If the user is saying goodbye or wants to end the conversation like 'bye jarvis.' respond with 'exit'.***
*** Respond with 'general (query)' if you can't decide the kind of query or if a query is asking to perform a task which is not mentioned above. ***
"""

```
ChatHistory = [
    {
        "role": "User", "message": "how are you?"
    },
    {
        "role": "Chatbot", "message": "general how are you?"
    },
    {
        "role": "User", "message": "do you like pizza?"
    },
    {
        "role": "Chatbot", "message": "general do you like pizza?"
    },
    {
        "role": "User", "message": "open chrome and tell me about mahatma gandhi."
```

```
    {
        "role": "Chatbot", "message": "open chrome and tell me about mahatma gandhi."
    },
    {
        "role": "User", "message": "open chrome and firefox."
    },
    {
        "role": "Chatbot", "message": "open chrome, open firefox."
    },
    {
        "role": "User", "message": "what is today's date and by the way remind me that i have a
dancing performance on 5th aug at 11pm"
    },
    {
        "role": "Chatbot", "message": "general what is today's date, reminder 11:00pm 5th aug
dancing performance"
    },
    {
        "role": "User", "message": "chat with me."
    },
    {
        "role": "Chatbot", "message": "general chat with me."
    }

]

def FirstLayerDMM(prompt: str = "test"):
    # Add the user's query to the messages list.
    messages.append({"role": "user", "content": f"{prompt}"})

    # Create a streaming chat session with the Cohere model.
    stream = co.chat_stream(
        model=CohereModel,  # Specify the Cohere model to use.
        message=prompt,  # Pass the user's query.
        temperature=0.7,  # Set the creativity level of the model.
        chat_history=ChatHistory,  # Provide the predefined chat history for context.
        prompt_truncation="OFF",  # Ensure the prompt is not truncated.
        connectors=[],  # No additional connectors are used.
        preamble=preamble  # Pass the detailed instruction preamble.
    )

    # Initialize an empty string to store the generated response.
    response = ""

    # Iterate over events in the stream and capture text generation events.
    for event in stream:
        if event.event_type == "text-generation":
            response += event.text  # Append generated text to the response.

    # Remove newline characters and split responses into individual tasks.
    response = response.replace("\n", "")
```

```
    # Strip leading and trailing whitespaces from each task.
    response = [i.strip() for i in response]

    # Initialize an empty list to filter valid tasks.
    temp = []

    for task in response:

        for func in funcs:
            if task.startswith(func):
                temp.append(task)

    response = temp

    if "(query)" in response:
        newresponse = FirstLayerDMM(prompt=prompt)
        return newresponse
    else:
        return response


if __name__ == "__main__":
    while True:
        print(FirstLayerDMM(input(">>> ")))
```

5.3.2 Integrated Testing

This phase focused on the communication between the **React.js** client and the **Python** server. It ensured that data packets (voice triggers or text responses) moved smoothly across the API.

- **End-to-End Pipeline:** We tested the flow starting from a voice command in the browser, sent to the Python server, processed, and the resulting action (like opening a website) reflected back on the system.
- **Latency Testing:** Measuring the time taken from the end of a user's speech to the start of the assistant's response. The goal was to keep this under 2 seconds for a seamless experience.
- **API Handshaking:** Ensuring that the Flask/FastAPI server correctly receives requests from the React frontend without CORS (Cross-Origin Resource Sharing) errors.

5.3.3 Beta Testing

The assistant was deployed in a real-world environment for a group of 5 users to test its usability and the **Voice Biometrics** feature.

- **Security Testing:** We attempted to trigger the assistant using voices of unauthorized users to verify if the "Unique Vocal Signature" biometric system successfully rejected them.

- **Environmental Testing:** Testing the assistant in different noise conditions (e.g., a quiet room vs. a room with a fan running) to calibrate the microphone sensitivity.

5.4 Modifications and Improvements

During the development and testing cycles, several technical challenges were identified. To meet the project objectives of high accuracy and a "User-Friendly" interface, the following modifications and improvements were implemented:

5.4.1 Ambient Noise Calibration

- **Initial Problem:** In the early stages, the Python backend would often fail to recognize commands or trigger "Ghost Commands" in environments with background noise (like a humming fan).
- **Modification:** We integrated a dynamic **Energy Threshold** adjustment. Before the assistant starts listening, it now runs a recognizer.adjust_for_ambient_noise function for 0.5 seconds to calibrate itself to the surrounding room volume.

5.4.2 Latency Reduction in React-Python Link

- **Initial Problem:** Using standard REST API calls for every voice segment caused a noticeable 3-4 second delay between the user speaking and the assistant responding.
- **Modification:** We improved the **Communication Protocol** by implementing "Early Triggering." Instead of waiting for the entire audio file to upload, the React frontend now sends a signal as soon as speech is detected, allowing the Python backend to "warm up" the recognition engine.

5.4.3 Voice Biometric Refinement

- **Initial Problem:** The system was initially too strict, sometimes rejecting the authorized user if they had a cold or a slightly different tone.
- **Improvement:** We implemented a **Confidence Score** system. Instead of a binary "Yes/No" for voice matching, the system now calculates a similarity percentage ($>85\%$). If the score is high enough, the command is executed, making the system more reliable for daily use while maintaining security.

5.4.4 Visual Feedback and UX Enhancements

- **Initial Problem:** Users were often confused about whether the assistant was actually "listening" or "thinking" because the screen was static.
- **Modification:** In the **React.js frontend**, we added a **State-Driven Animation** (a pulse effect).
    - **Blue Pulse:** Assistant is listening.
    - **Rotating Ring:** Assistant is processing (API call in progress).
    - **Green Wave:** Assistant is speaking (TTS active).

5.4.5 Fallback Mechanism

- **Improvement:** We added a "Type-to-Command" fallback in the React interface. If the environment is too noisy for voice recognition, the user can manually type the command into a text box, ensuring the assistant remains functional under all conditions.

5.5 Test Cases

The following test cases describe the specific scenarios used to validate the functional requirements of the AI Voice Assistant. Each test ensures that the "Speech-to-Do" objective is met while maintaining system security.

Test ID	Test Case Description	Input / Action	Expected Output	Status
TC-01	**Authorized User Access**	Authorized user speaks: "Wake up."	System matches voice signature and activates the listener.	**Pass**
TC-02	**Unauthorized User Access**	Different person speaks: "Open My Files."	System identifies a signature mismatch and denies access.	**Pass**
TC-03	**Speech-to-Text (STT) Accuracy**	User says: "Search for Mars on Wikipedia."	React UI displays the correct text: "Searching Wikipedia for Mars..."	**Pass**
TC-04	**System Command Execution**	User says: "What is the time?"	Python backend fetches system time and speaks it via TTS.	**Pass**
TC-05	**Application Launching**	User says: "Open Google Chrome."	The webbrowser module triggers and opens the browser.	**Pass**
TC-06	**Frontend-Backend Sync**	User clicks 'Microphone' in React.	The Python script immediately enters 'Listen' mode.	**Pass**
TC-07	**Ambient Noise Handling**	User speaks in a room with a running fan.	Assistant filters noise and correctly identifies the command.	**Pass**
TC-08	**Text-to-Speech (TTS) Feedback**	System completes a task.	Assistant says "Task Completed" or "Here is what I found."	**Pass**

TC-09	**Fallback Mechanism**	User types "Open Notepad" in React UI.	System opens Notepad without requiring voice input.	**Pass**

Test ID	**Test Case Description**	**Input / Action**	**Expected Output**	**Status**
TC-10	**Session Termination**	User says: "Exit" or "Stop."	The Python process clears the cache and stops the listener.	**Pass**

CHAPTER 6

RESULTS AND DISCUSSION

Results Overview

The development of the AI Voice Assistant resulted in a fully functional, cross-platform application. The system successfully integrates a React.js frontend with a Python backend, achieving the primary goal of "Speech-to-Perform" task execution. The voice biometric security layer ensures that the system remains responsive only to the primary user, providing a personalized and secure experience.

Discussion

The results indicate that a decoupled architecture (React + Python) is highly effective for building AI assistants.

- **User Interface vs. Logic:** By using **React.js**, we provided a visual experience that standard "command-line" assistants lack. The real-time status updates (Listening, Processing) reduced user anxiety during the short processing waits.

- **Python's Versatility:** The vast ecosystem of Python libraries allowed us to implement STT, TTS, and OS-level automation without building models from scratch, significantly speeding up the development lifecycle.

- **The Biometric Challenge:** While the voice signature adds security, it also adds a layer of complexity. We found that environmental factors (echo, microphone quality) play a significant role in biometric accuracy, suggesting that a high-quality microphone is essential for this project.

6.1 Test Reports

The testing phase was conducted to validate the integration between the **React.js Frontend** and the **Python Backend**. The following report summarizes the performance metrics and functional validation.

6.1.1 Functional Validation Report

We performed a series of "Stress Tests" to see how the assistant handles rapid commands.

- **Command Success Rate:** Out of 50 varied voice commands (Open App, Search Wikipedia, Tell Time), **46 were executed successfully** on the first attempt.

- **Biometric Security Check:** 20 attempts were made by unauthorized voices; the system correctly denied access in **19 cases**, showing a **95% Security Success Rate**.

6.1.2 Performance Metrics (Average)

Parameter	Measurement	Observation
Boot-up Time	3.5 Seconds	Time taken for Flask/FastAPI and React to initialize.
STT Latency	1.1 Seconds	Time to convert Speech to Text.
Action Execution	0.4 Seconds	Time to trigger system commands (e.g., Opening Notepad).
CPU Usage	12% - 18%	Low resource consumption on a standard i5 processor.

6.1.3 Error Log Summary

During testing, the most common error was **"Speech Recognition Timeout"** (Error 408),

which occurred when the user did not speak within 5 seconds of clicking the mic. We resolved this by adding a "Speak Now" visual cue in the React UI.

6.2 User Documentation

This section serves as a guide for the end-user to install and operate the AI Voice Assistant.

6.2.1 System Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Language Environment:** Python 3.8 or higher.
- **Frontend Tools:** Node.js (for React environment).
- **Hardware:** Functional Microphone and Internet Connection (for API-based searches).

6.2.2 Setup and Installation

1. **Clone the Repository:** Download the project folder to your local machine.
2. **Backend Setup:**
    - Navigate to the /backend folder.
    - Run pip install -r requirements.txt to install libraries like SpeechRecognition, pyttsx3, and Flask.
3. **Frontend Setup:**
    - Navigate to the /frontend folder.
    - Run npm install followed by npm start.
4. **Run the Application:** Start the Python server first, then launch the React dashboard.

6.2.3 Operating Instructions

- **Voice Enrollment:** On the first run, the user must record their voice for 5 seconds to set up the **Voice Biometric** signature.
- **Triggering the Assistant:** Click the **"Microphone"** icon on the React Dashboard. When the pulse animation turns **Blue**, speak your command.
- **Example Commands:**
    - *"Hey Assistant, open YouTube."*
    - *"Search Wikipedia for Artificial Intelligence."*
    - *"What time is it?"*
- **Stopping the Assistant:** Say *"Exit"* or click the **"Stop"** button on the UI to terminate the session safely.

6.2.4 Troubleshooting

- **Assistant not responding?** Check if the microphone is being used by another app (like Zoom or Teams).
- **Wrong commands?** Ensure you are in a quiet environment and have calibrated the "Ambient Noise" settings in the app.

CHAPTER 7

CONCLUSIONS

7.1 Conclusion

The development of the AI Voice Assistant successfully demonstrates the integration of modern web technologies (React.js) with powerful scripting capabilities (Python). The project achieved its primary objective of creating a "Speech-to-Do" system that allows users to perform system tasks, web searches, and information retrieval through natural language. By shifting the interface from a traditional CLI to a dynamic React dashboard, the system bridges the gap between complex AI logic and user-centric design.

7.1.1 Significance of the System

The significance of this AI Voice Assistant lies in three key areas:

- **Hands-Free Productivity:** It provides an accessible solution for users to multitask or for individuals with physical disabilities who may find manual typing challenging.
- **Security through Biometrics**: Unlike standard open-access assistants, the integration of Voice Biometrics ensures that the system remains a personal tool, responding only to the authorized user's unique vocal signature.
- **Hybrid Architecture**: The project proves that using a Decoupled Architecture (React Frontend + Python Backend) allows for a scalable and responsive AI application that can run on standard consumer hardware without heavy cloud dependency.

7.2 Limitations of the System

Despite the successful implementation, certain limitations were observed during the testing phase:

- **Internet Dependency:** While many system commands work offline, features like Wikipedia search and advanced Speech-to-Text (STT) still require an active internet connection for high accuracy.
- **Environmental Sensitivity:** High levels of ambient noise or low-quality hardware (microphones) can occasionally interfere with the **Voice Biometric** matching, leading to false rejections.
- **Fixed Command Vocabulary:** Currently, the assistant relies on specific keyword mapping. It may struggle with highly abstract or conversational "small talk" that falls outside its programmed intent-recognition logic.
- **Resource Latency:** On older hardware, the simultaneous running of a React development server and a Python backend can consume significant RAM (>800 MB)

7.3 Future Scope of the Project

The current version serves as a robust foundation that can be expanded in the following ways:

- **Large Language Model (LLM) Integration:** Integrating APIs like GPT-4 or local models (Llama 3) would allow the assistant to handle complex reasoning, summarization, and natural conversations.
- **Home Automation (IoT):** The Python backend can be extended to control smart home devices (Lights, Fans, Thermostats) using protocols like MQTT or Zigbee.
- **Multilingual Support:** Future versions could include NLP libraries that support regional languages (like Hindi, Spanish, or Arabic) to make the technology more

- **Face Recognition Integration**: Adding a visual security layer alongside voice biometrics would create a "Multi-Factor Bio-Authentication" system for sensitive tasks.
- **Mobile Application:** Converting the React frontend into a React Native app would allow the assistant to be used on smartphones, providing a truly portable AI experience.

REFERENCES

- **React Documentation. (2026).** React – A JavaScript library for building user interfaces. Retrieved from https://react.dev/

- **Python Software Foundation.** Python Language Reference, version 3.x. Available at: https://docs.python.org/3/

- **Zhang, A. (2023).** *SpeechRecognition:* Library for performing speech recognition, with support for several engines and APIs, online and offline. PyPI. Available at: https://pypi.org/project/SpeechRecognition/

- **Cambre, J. (2022).** pyttsx3: Text-to-Speech (TTS) library for Python 2 and 3. PyPI. Available at: https://pypi.org/project/pyttsx3/

- **Flask/FastAPI Documentation.** Web Server Gateway Interface (WSGI) / Asynchronous Server Gateway Interface (ASGI) for Python. Available at: https://flask.palletsprojects.com/

- **GeeksforGeeks.** Build a Virtual Assistant using Python. (For implementation logic and command mapping).

- **Medium - Towards Data Science.** Voice Biometrics and Audio Signal Processing in Python. (For MFCC and Vocal Signature concepts).